

# Overview of Deep Learning: Key Algorithms

## Summer School – Exposome Analytics

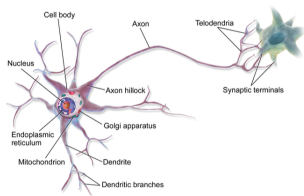
Prof. Benoit Lique

LMAP, Université de Pau et des Pays de l'Adour (France)

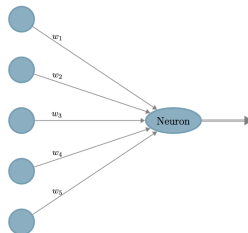
Thursday 17 September 2026

**EXPOSOME ANALYTICS**, Summer School, 14–18 September 2026  
Imperial School of Public Health, London, United Kingdom

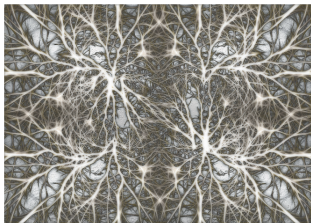
# Biological vs. Artificial Neurons: The Building Blocks of Neural Networks



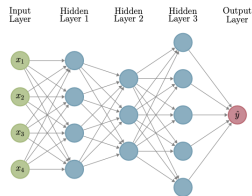
(a)



(b)



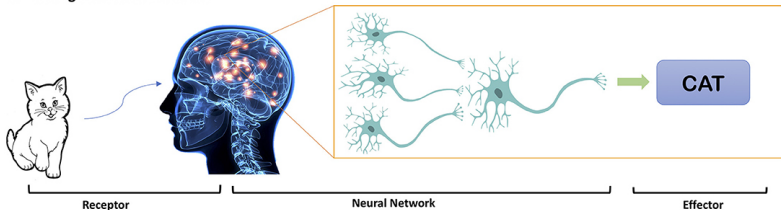
(c)



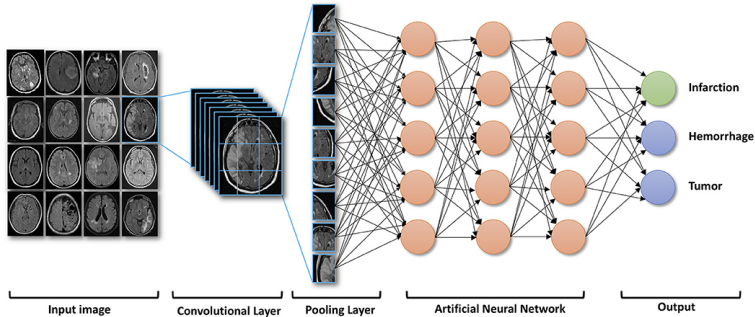
(d)

# Brain Inspiration: CNNs for Medical Imaging

## A Biological Neural Network



## B Computer Neural Network(Convolutional Neural Network)



# A Taste of Tasks and Architectures

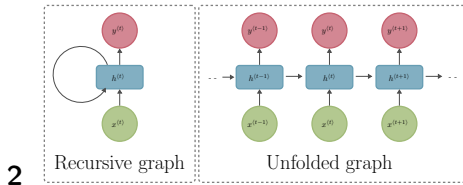
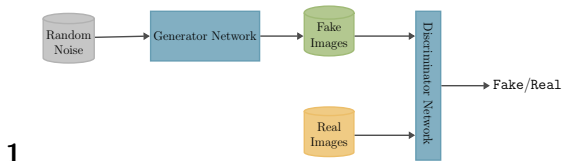
Eight families of models, eight pictures.

*Which picture goes with which name?*

- ▶ Fully Connected Neural Networks
- ▶ Convolutional Neural Networks
- ▶ Recurrent Neural Networks, LSTM
- ▶ Transformer architectures
- ▶ Encoder, Decoder, Auto-encoders
- ▶ Generative Adversarial Networks
- ▶ Diffusion models
- ▶ Deep Reinforcement Learning

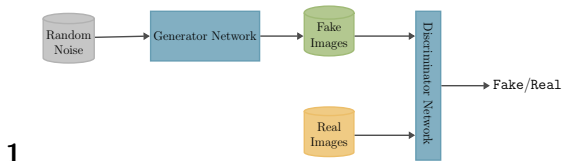
*By the end of the morning, the first four will no longer be black boxes.*

# Which Architecture? (1/4)

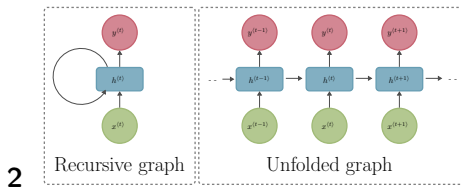


Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Architecture? (1/4)

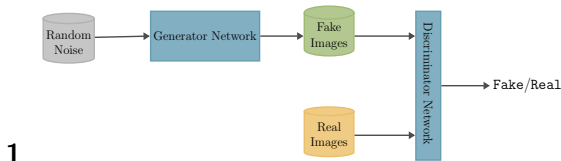


## Generative Adversarial Networks

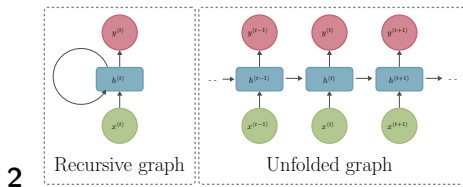


Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Architecture? (1/4)



## Generative Adversarial Networks

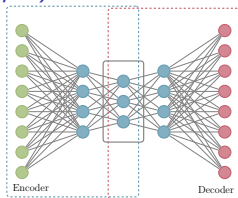


## Recurrent Neural Networks, LSTM

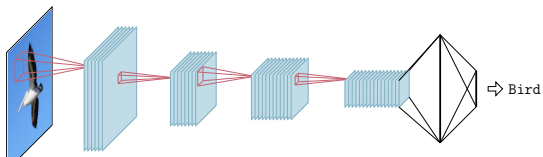
Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

## Which Architecture? (2/4)

3

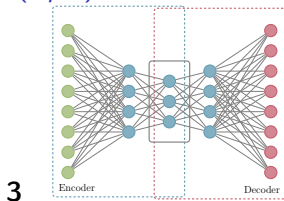


4

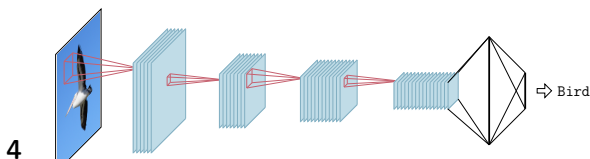


Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

## Which Architecture? (2/4)

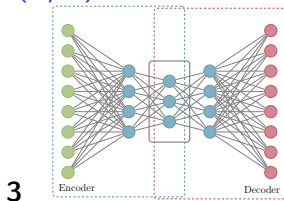


## Encoder, Decoder, Auto-encoder models

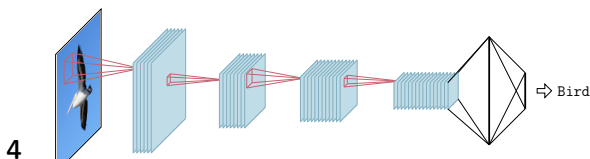


Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

## Which Architecture? (2/4)



## Encoder, Decoder, Auto-encoder models

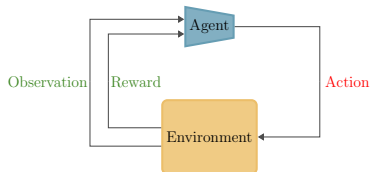


## Convolutional Neural Networks

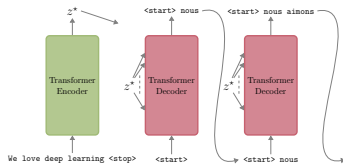
Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Architecture? (3/4)

5

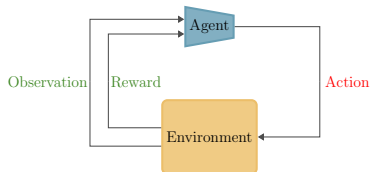


6



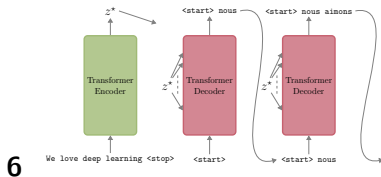
Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Architecture? (3/4)



5

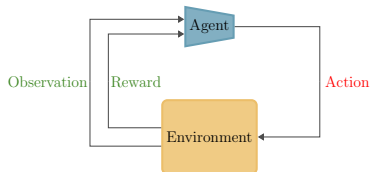
## Deep Reinforcement Learning



6

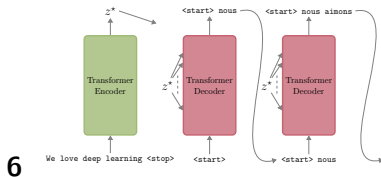
Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Architecture? (3/4)



5

## Deep Reinforcement Learning



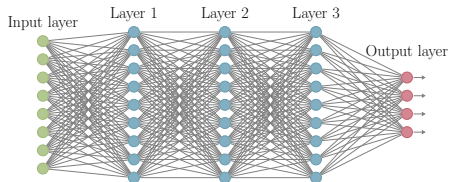
6

## Transformer architectures

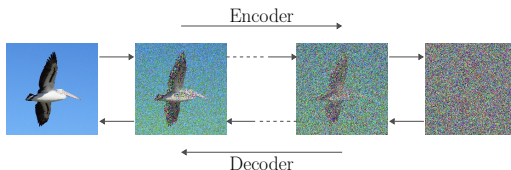
Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Architecture? (4/4)

7

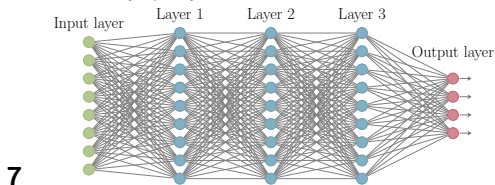


8

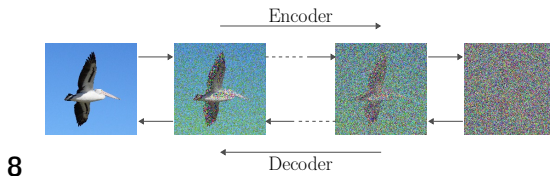


Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Architecture? (4/4)

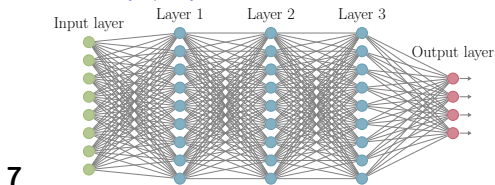


## Fully Connected Neural Networks

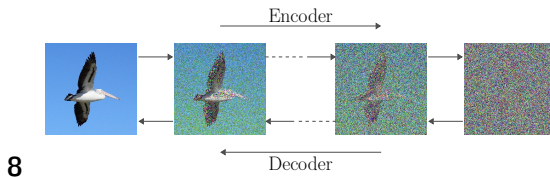


Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Architecture? (4/4)



## Fully Connected Neural Networks



## Diffusion models

Fully connected · Convolutional · Recurrent, LSTM · Transformer · Auto-encoder · GAN ·  
Diffusion · Reinforcement learning

# Which Task for Which Architecture?

| Architecture           | Typical task                                     |
|------------------------|--------------------------------------------------|
| Fully connected        | tabular data, a vector in, a prediction out      |
| Convolutional          | images, signals, anything with a local structure |
| Recurrent, LSTM        | sequences: text, time series, signals            |
| Transformer            | sequences, but all at once: language models      |
| Auto-encoder           | compress, denoise, detect anomalies              |
| GAN, Diffusion         | generate new images, molecules, data             |
| Reinforcement learning | decide, act, receive a reward                    |

# Which Task for Which Architecture?

| Architecture           | Typical task                                     |
|------------------------|--------------------------------------------------|
| Fully connected        | tabular data, a vector in, a prediction out      |
| Convolutional          | images, signals, anything with a local structure |
| Recurrent, LSTM        | sequences: text, time series, signals            |
| Transformer            | sequences, but all at once: language models      |
| Auto-encoder           | compress, denoise, detect anomalies              |
| GAN, Diffusion         | generate new images, molecules, data             |
| Reinforcement learning | decide, act, receive a reward                    |

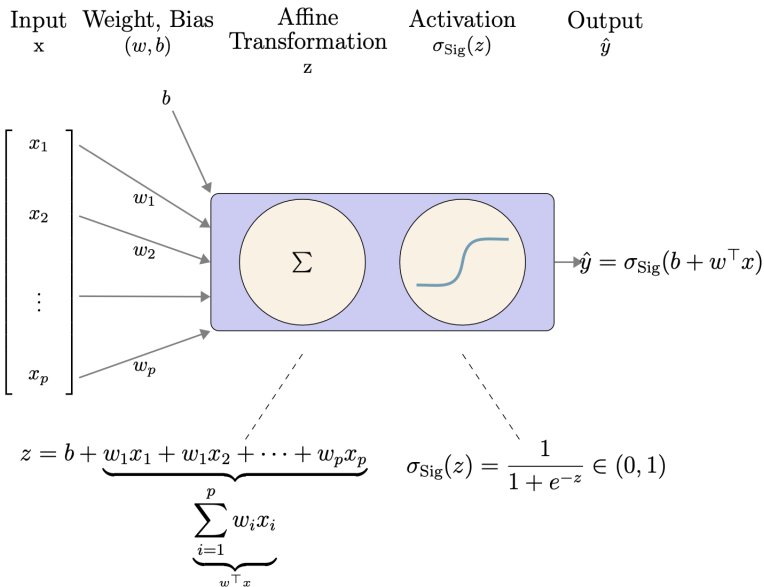
Today we open the box behind the first four.

## Question 1

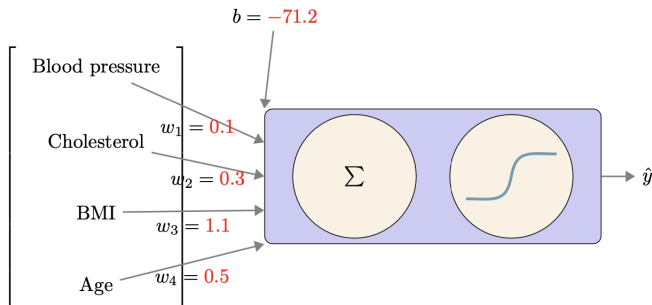
*How confident are you with the math or algebra behind deep learning models?*



# Artificial neuron: Sigmoid model (logistic model)



# Toy example: predict stroke



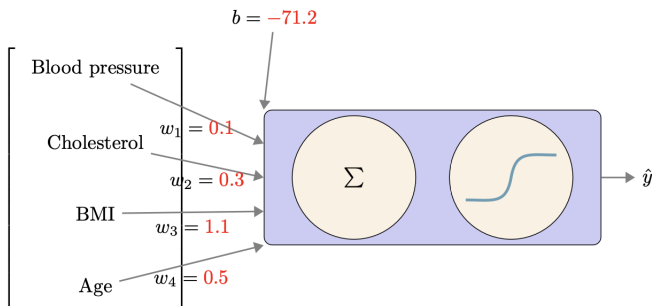
Patient 1

$$x = \begin{bmatrix} 130 \\ 6 \\ 28 \\ 55 \end{bmatrix}$$

$$z = \underbrace{-71.2}_b + \underbrace{0.1}_{w_1} \cdot \underbrace{130}_{x_1} + \underbrace{0.3}_{w_2} \cdot \underbrace{6}_{x_2} + \underbrace{1.1}_{w_3} \cdot \underbrace{28}_{x_3} + \underbrace{0.5}_{w_4} \cdot \underbrace{55}_{x_4} = 1.9$$

$$\Rightarrow \hat{y} = \sigma_{\text{Sig}}(z) = \frac{1}{1 - e^{-1.9}} = \underbrace{0.71}_{\text{Probability of getting a stroke}}$$

# Toy example: predict stroke



Patient 2

$$x = \begin{bmatrix} 130 \\ 5.5 \\ 23 \\ 56 \end{bmatrix}$$

$$z = \underbrace{-71.2}_b + \underbrace{0.1}_{w_1} \cdot \underbrace{130}_{x_1} + \underbrace{0.3}_{w_2} \cdot \underbrace{5.5}_{x_2} + \underbrace{1.1}_{w_3} \cdot \underbrace{23}_{x_3} + \underbrace{0.5}_{w_4} \cdot \underbrace{56}_{x_4}$$
$$= -3.25$$

$$\Rightarrow \hat{y} = \sigma_{\text{Sig}}(z) = \frac{1}{1 - e^{-(-3.25)}} = \underbrace{0.04}_{\text{Probability of getting a stroke}}$$

# Sigmoid model as a binary classifier

A classifier can be constructed via a *decision rule* based on a threshold  $\tau$ :

$$\widehat{y} = \begin{cases} 0 \text{ (negative),} & \text{if } \hat{y} \leq \tau \\ 1 \text{ (positive),} & \text{if } \hat{y} > \tau. \end{cases}$$

In many cases one selects the threshold at  $\tau = 0.5$ .

# Sigmoid model as a binary classifier

A classifier can be constructed via a *decision rule* based on a threshold  $\tau$ :

$$\widehat{\mathcal{Y}} = \begin{cases} 0 \text{ (negative),} & \text{if } \hat{y} \leq \tau \\ 1 \text{ (positive),} & \text{if } \hat{y} > \tau. \end{cases}$$

In many cases one selects the threshold at  $\tau = 0.5$ .

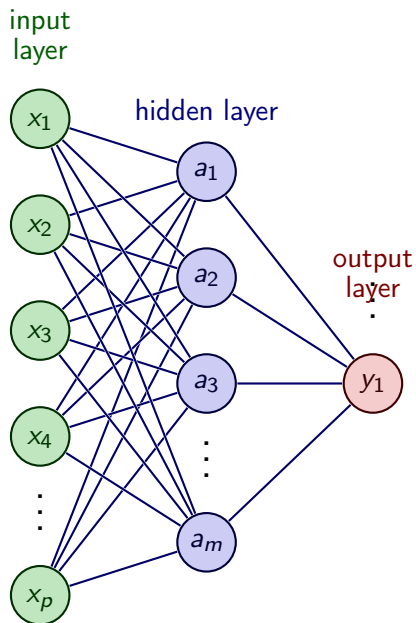
- ▶ Patient 1:  $\hat{y} = 0.71 > 0.5 \Rightarrow \widehat{\mathcal{Y}} = 1$  (Positive)
- ▶ Patient 2:  $\hat{y} = 0.04 \leq 0.5 \Rightarrow \widehat{\mathcal{Y}} = 0$  (Negative)

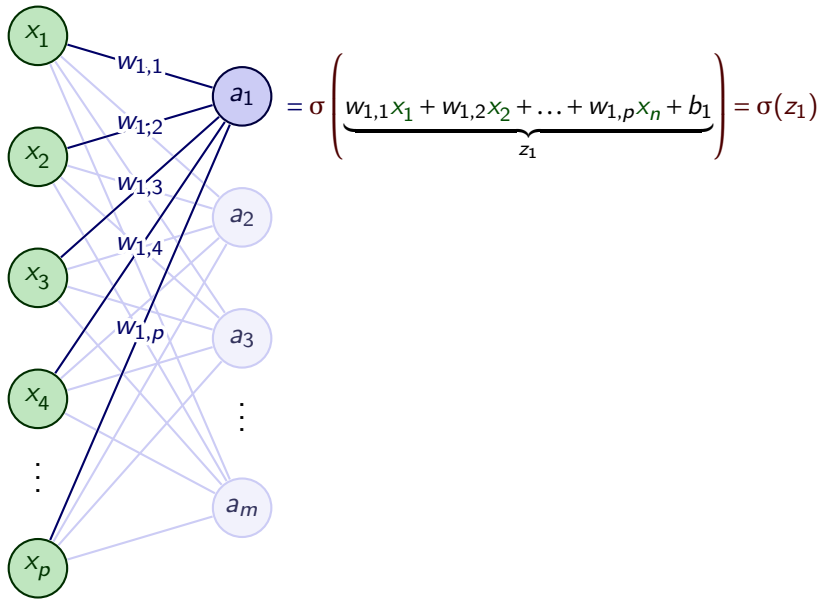
## Question 2

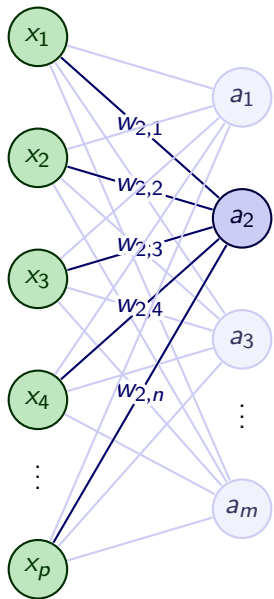
*Will changing the threshold after applying the sigmoid function affect the model's performance?*



# Single hidden layer

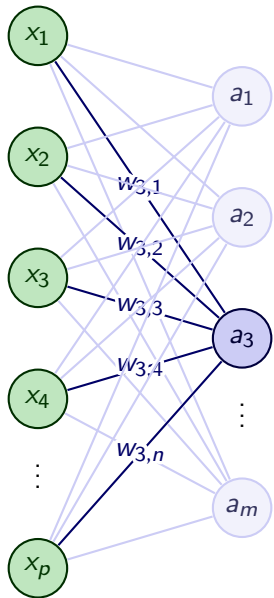






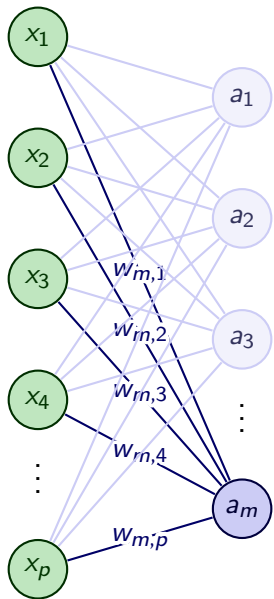
$$a_2 = \sigma \left( \underbrace{w_{2,1}x_1 + w_{2,2}x_2 + \dots + w_{2,p}x_n + b_2}_{z_2} \right)$$

$$= \sigma(z_2)$$



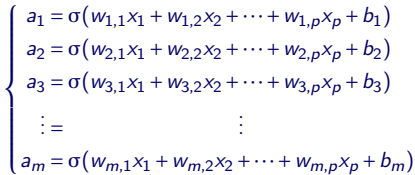
$$a_3 = \sigma \left( \underbrace{w_{3,1}x_1 + w_{3,2}x_2 + \dots + w_{3,p}x_n + b_3}_{z_3} \right)$$

$$= \sigma(z_3)$$



$$a_m = \sigma \left( \underbrace{w_{m,1}x_1 + w_{m,2}x_2 + \dots + w_{m,p}x_p}_{z_m} + b_m \right)$$

$$= \sigma(z_m)$$





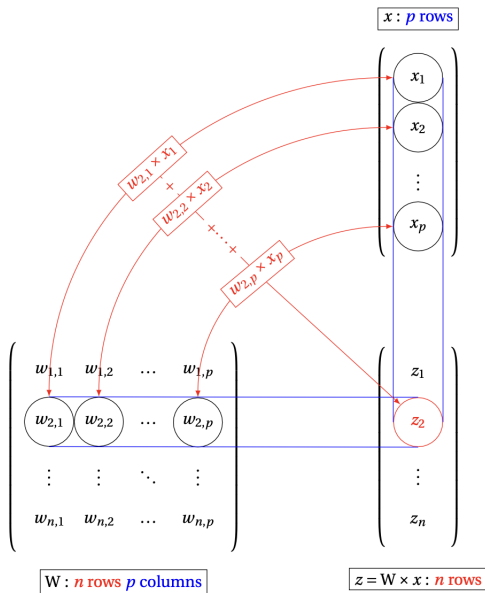
# Matrix notation

Vectorized form of neuron activations:

$$a = \sigma(Wx + b)$$

$$\begin{aligned} \overbrace{\begin{pmatrix} a_1 \\ \vdots \\ a_m \end{pmatrix}}^a &= \sigma \left( \underbrace{\overbrace{\begin{pmatrix} w_{1,1} & \cdots & w_{1,p} \\ \vdots & \ddots & \vdots \\ w_{m,1} & \cdots & w_{m,p} \end{pmatrix}}^W}_{z} \underbrace{\overbrace{\begin{pmatrix} x_1 \\ \vdots \\ x_p \end{pmatrix}}^{\text{Input } x}} + \underbrace{\overbrace{\begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}}^b} \right) \\ &= \sigma \begin{pmatrix} z_1 \\ \vdots \\ z_p \end{pmatrix} = \begin{pmatrix} \sigma(z_1) \\ \vdots \\ \sigma(z_p) \end{pmatrix} \end{aligned}$$

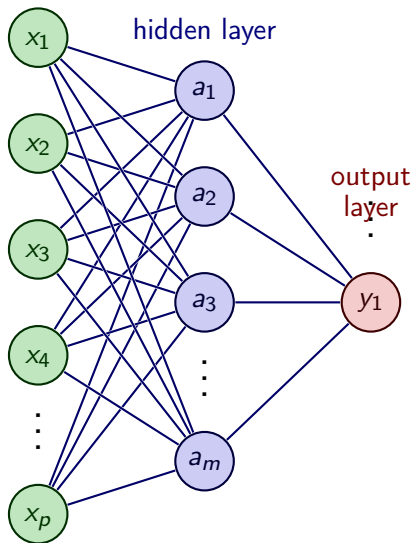
# Matrix-vector multiplication: $W \times x = z$ with $b = 0$



input  
layer

hidden layer

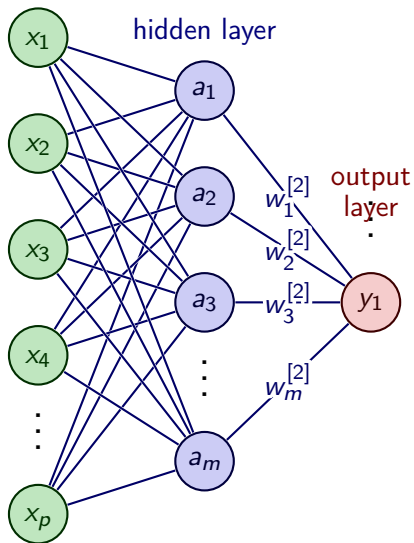
output  
layer



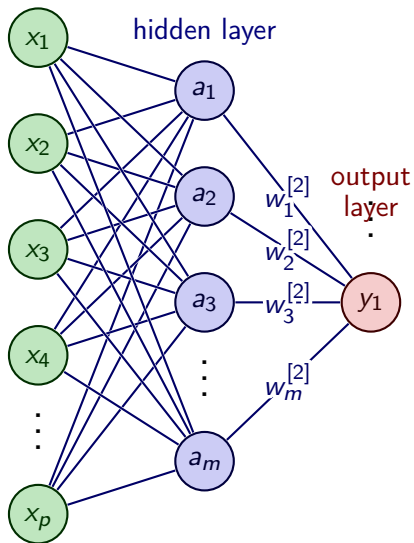
input  
layer

hidden layer

output  
layer

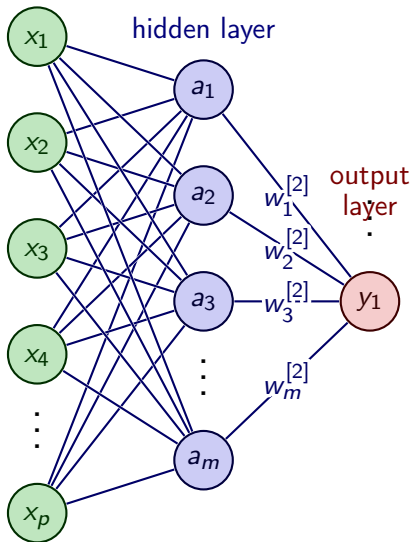


input  
layer



$$y = \sigma(w_1^{[2]} a_1 + w_2^{[2]} a_2 + \dots + w_m^{[2]} a_m + b^{[2]})$$
$$= \sigma(W^{[2]} a + b^{[2]})$$

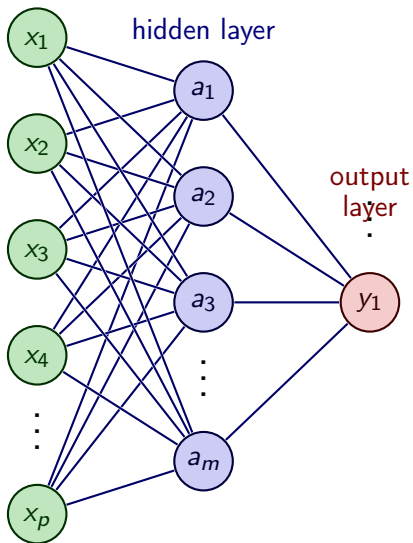
input  
layer



$$y = \sigma \left( \overbrace{w_1^{[2]} a_1 + w_2^{[2]} a_2 + \dots + w_m^{[2]} a_m}^{W^{[2]} a} + b^{(2)} \right)$$
$$= \sigma \left( W^{[2]} a + b^{(2)} \right)$$

$$W^{[2]} a = \overbrace{\begin{pmatrix} w_1^{[2]} & w_2^{[2]} & \dots & w_m^{[2]} \end{pmatrix}}^{W^{[2]}} \overbrace{\begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{pmatrix}}^a$$
$$= \left( w_1^{[2]} a_1 + w_2^{[2]} a_2 + \dots + w_m^{[2]} a_m \right)$$

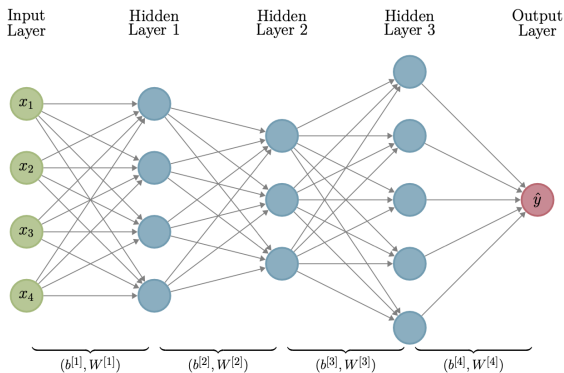
input  
layer



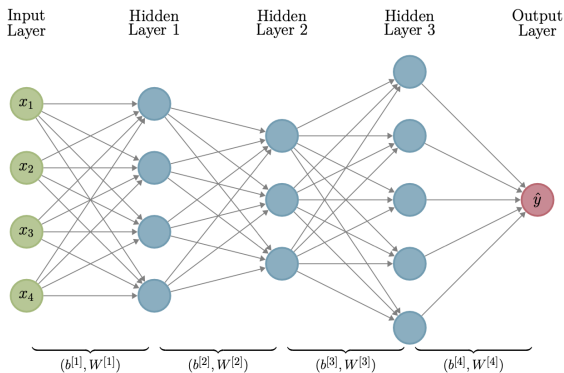
$$y = \sigma \left( W^{[2]} \underbrace{(\sigma(W^{[1]}x + b^{[1]}))}_{a^{[1]}} + b^{[2]} \right)$$
$$= \underbrace{f^{[2]}(f^{[1]}(x))}_{f_0(x)}$$

where  $\theta = (W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]})$

# Fully Connected Neural Network or feedforward networks

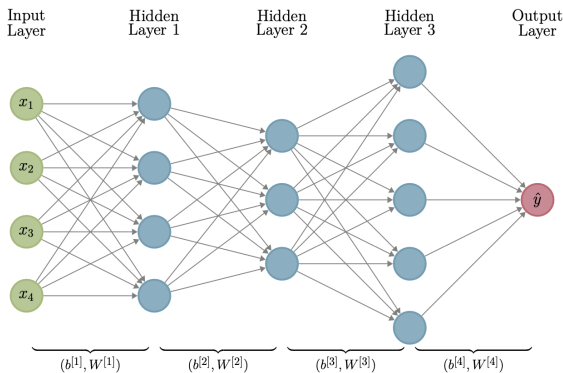


# Fully Connected Neural Network or feedforward networks



**Composition of function :**  $\hat{y} = f^{[L]}(f^{[L-1]}(f^{[L-2]}(\dots(f^{[1]}(x))\dots)))$

# Fully Connected Neural Network or feedforward networks



**Composition of function :**  $\hat{y} = f^{[L]}(f^{[L-1]}(f^{[L-2]}(\dots(f^{[1]}(x))\dots)))$

$$\hat{y} = \underbrace{\sigma^{[4]}(W^{[4]} \underbrace{\sigma^{[3]}(W^{[3]} \underbrace{\sigma^{[2]}(W^{[2]} \underbrace{\sigma^{[1]}(W^{[1]}x + b^{[1]})}_{f^{[1]}}) + b^{[2]}}_{f^{[2]}}) + b^{[3]}}_{f^{[3]}}) + b^{[4]}}_{f^{[4]}}$$

## Question 3

*Does a fully connected neural network with one hidden layer allow interactions between input variables?*



# Why Single Hidden Layer is Useful?

## Universal Approximation Theorem

Consider a continuous function  $f^* : \mathcal{K} \rightarrow \mathbb{R}^q$  where  $\mathcal{K} \subset \mathbb{R}^p$  is a compact set. Then for any any  $\sigma(\cdot)$  non-polynomial activation function and any  $\varepsilon > 0$  there exists an  $N_1$  and parameters  $W^{[1]} \in \mathbb{R}^{N_1 \times p}$ ,  $b^{[1]} \in \mathbb{R}^{N_1}$ , and  $W^{[2]} \in \mathbb{R}^{q \times N_1}$ , such that the function

$$f_{\theta}(x) = W^{[2]} \sigma(W^{[1]}x + b^{[1]}),$$

satisfies  $\|f_{\theta}(x) - f^*(x)\| < \varepsilon$  for all  $x \in \mathcal{K}$ .

# Why Single Hidden Layer is Useful?

## Universal Approximation Theorem

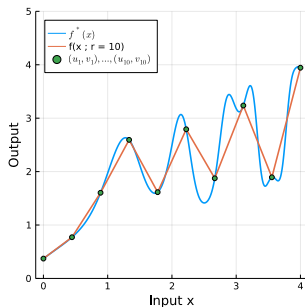
Consider a continuous function  $f^* : \mathcal{X} \rightarrow \mathbb{R}^q$  where  $\mathcal{X} \subset \mathbb{R}^p$  is a compact set. Then for any  $\sigma(\cdot)$  non-polynomial activation function and any  $\varepsilon > 0$  there exists an  $N_1$  and parameters  $W^{[1]} \in \mathbb{R}^{N_1 \times p}$ ,  $b^{[1]} \in \mathbb{R}^{N_1}$ , and  $W^{[2]} \in \mathbb{R}^{q \times N_1}$ , such that the function

$$f_\theta(x) = W^{[2]} \sigma(W^{[1]}x + b^{[1]}),$$

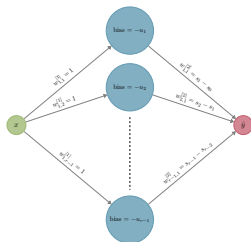
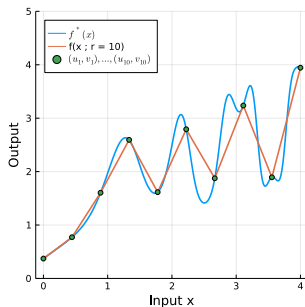
satisfies  $\|f_\theta(x) - f^*(x)\| < \varepsilon$  for all  $x \in \mathcal{X}$ .

**In simple words:** A neural network with just *one hidden layer* can, in principle, learn to approximate any smooth function, if it has *enough hidden neurons*.

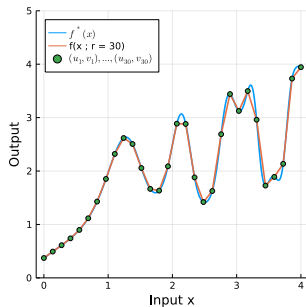
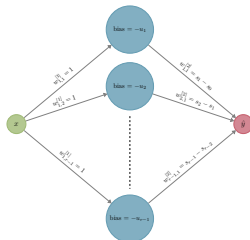
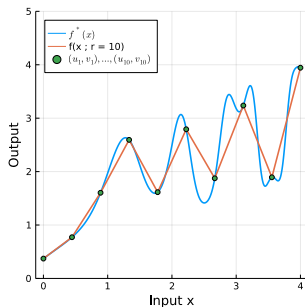
# Simple Function Approximation



# Simple Function Approximation

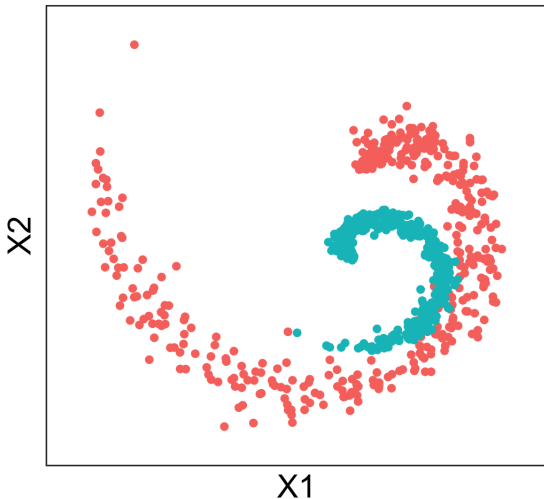


# Simple Function Approximation



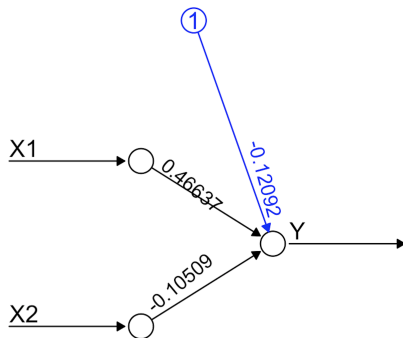
## Expressivity:

Binary classification task with two features:  $x_1$  and  $x_2$



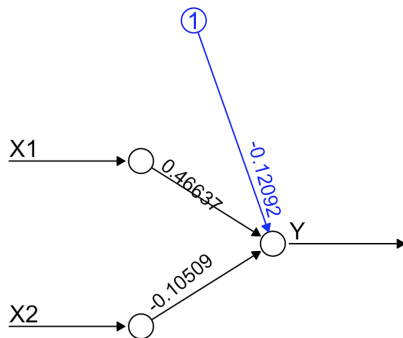
## Sigmoid model: no hidden layer

Total parameter: 2 (weights) + 1 (bias) = 3



# Sigmoid model: no hidden layer

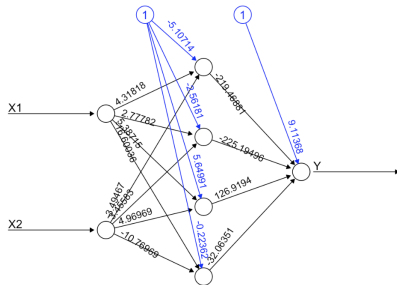
Total parameter: 2 (weights) + 1 (bias) = 3



# Expressivity: one hidden layer with 4 neurons

hidden Layer 1    output layer

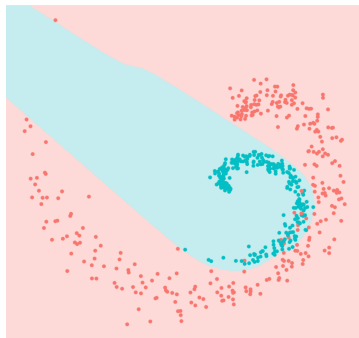
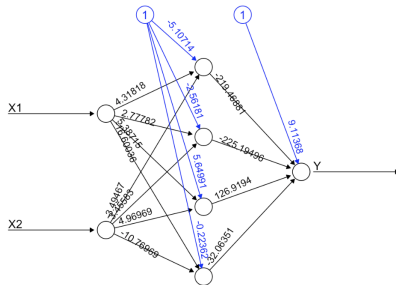
$$\text{Total parameter: } \overbrace{4 \times 2 + 4} + \overbrace{4 + 1} = 17$$



# Expressivity: one hidden layer with 4 neurons

hidden Layer 1    output layer

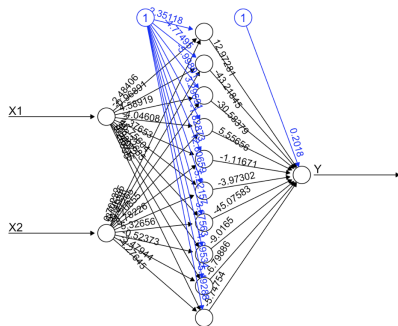
$$\text{Total parameter: } \overbrace{4 \times 2 + 4} + \overbrace{4 + 1} = 17$$



# Expressivity: one hidden layer with 10 neurons

hidden Layer 1    output layer

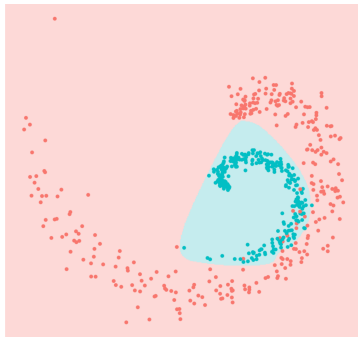
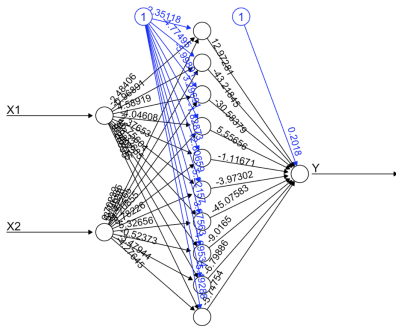
$$\text{Total parameter: } \overbrace{10 \times 2 + 10} + \overbrace{10 + 1} = 41$$



# Expressivity: one hidden layer with 10 neurons

hidden Layer 1    output layer

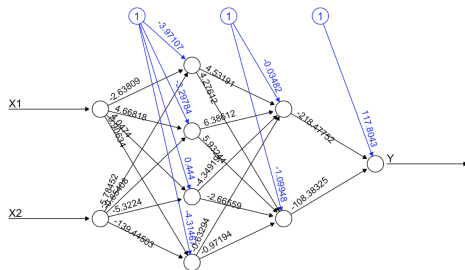
$$\text{Total parameter: } \overbrace{10 \times 2 + 10} + \overbrace{10 + 1} = 41$$



# Why not going deep: 2 hidden layers

hidden Layer 1    hidden Layer 2    output layer

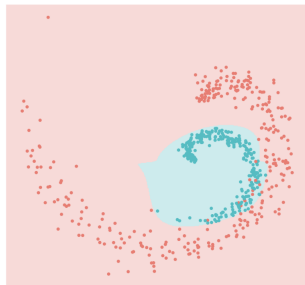
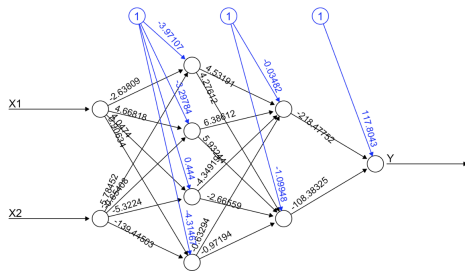
Total parameter:  $\overbrace{4 \times 2 + 4} + \overbrace{2 \times 4 + 2} + \overbrace{2 + 1} = 25$



# Why not going deep: 2 hidden layers

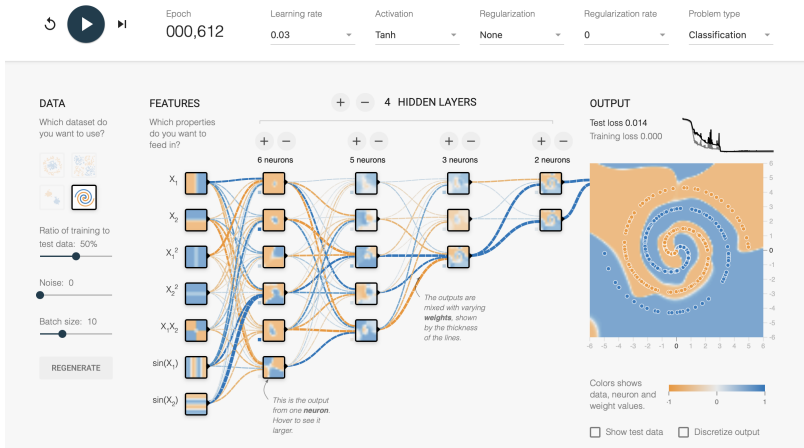
hidden Layer 1    hidden Layer 2    output layer

Total parameter:  $\overbrace{4 \times 2 + 4} + \overbrace{2 \times 4 + 2} + \overbrace{2 + 1} = 25$



# Short demo

<https://playground.tensorflow.org>



# Why Go Deep?

- ▶ A single hidden layer can approximate any function
- ▶ But it may require **a huge number of neurons**

# Why Go Deep?

- ▶ A single hidden layer can approximate any function
- ▶ But it may require **a huge number of neurons**
- ▶ Deep networks build functions **hierarchically**

$$\hat{y} = f^{[L]}(f^{[L-1]}(f^{[L-2]}(\dots(f^{[1]}(x))\dots)))$$

- ▶ Each layer learns more abstract features

# Shallow vs Deep Representations

## Shallow network

- ▶ Many neurons
- ▶ Complex decision boundary learned at once

## Deep network

- ▶ Fewer neurons per layer
- ▶ Decision boundary built step by step

*Depth allows feature reuse and composition.*

# Expressivity of Deep Networks

## Key Result

There exist functions that:

- ▶ require **exponentially many neurons** with one hidden layer
  - ▶ can be represented with **polynomial size** deep networks
- 
- ▶ Telgarsky (2016): *Benefits of depth in neural networks*
  - ▶ Eldan & Shamir (2016): *The power of depth for feedforward networks*

## Question 4

*Does the fact that a neural network can approximate very complex functions guarantee that we can easily train it?*

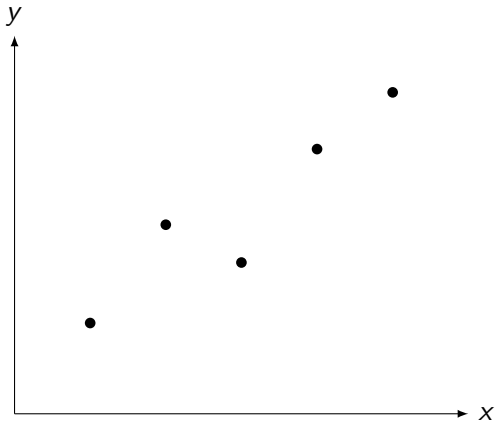


# From Expressivity to Learning

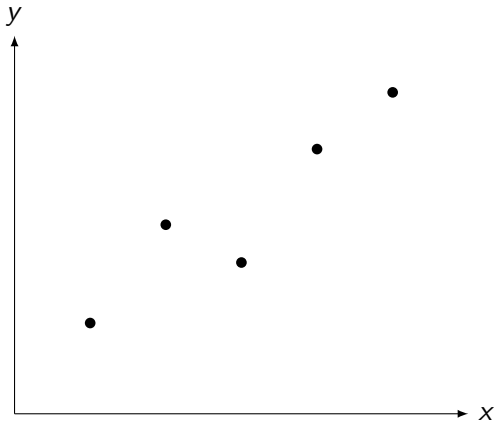
- ▶ Deep networks can represent very complex functions
- ▶ But representation alone is not enough
- ▶ We must **learn** the parameters from data

Learning = minimizing a **loss** (i.e. error) between predictions ( $\hat{y}$  and observed outcome  $y$ )

## One-parameter model and no bias ( $b = 0$ )



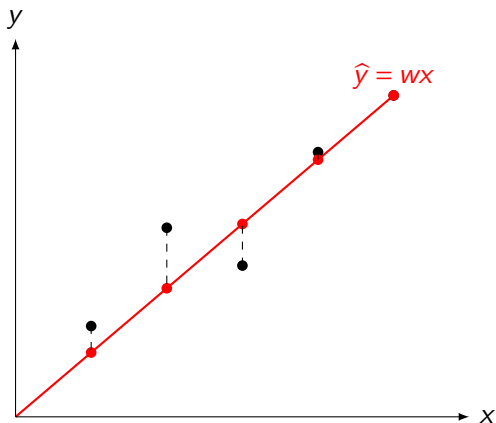
## One-parameter model and no bias ( $b = 0$ )



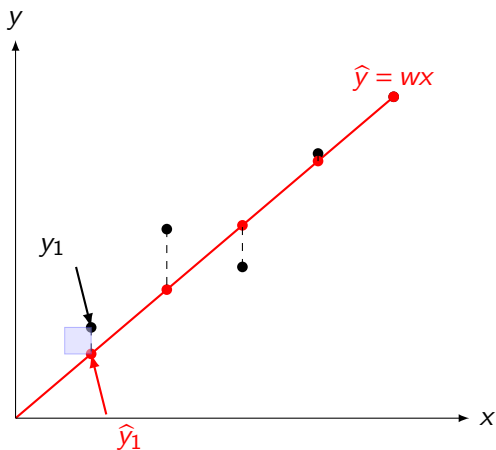
Simple linear model viewed as artificial neuron with a **linear activation function** ( $\sigma(u) = u$ ).

$$y = w \times x, \quad \text{which best value for } w?$$

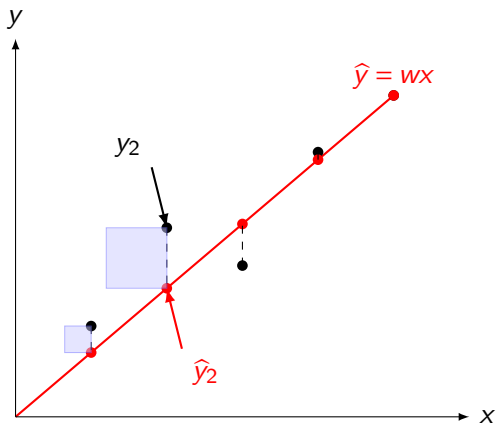
Find the best lines, i.e choose a suitable value for  $w$



Find the best lines, i.e choose a suitable value for  $w$

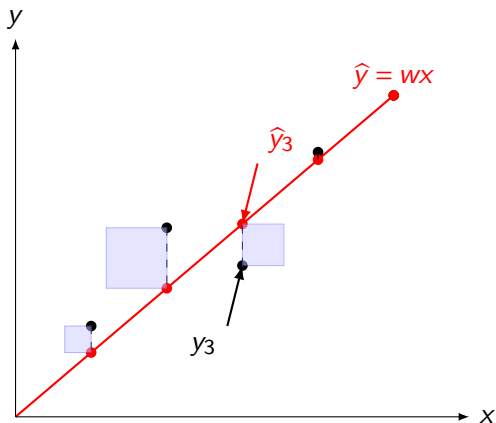


Find the best lines, i.e choose a suitable value for  $w$



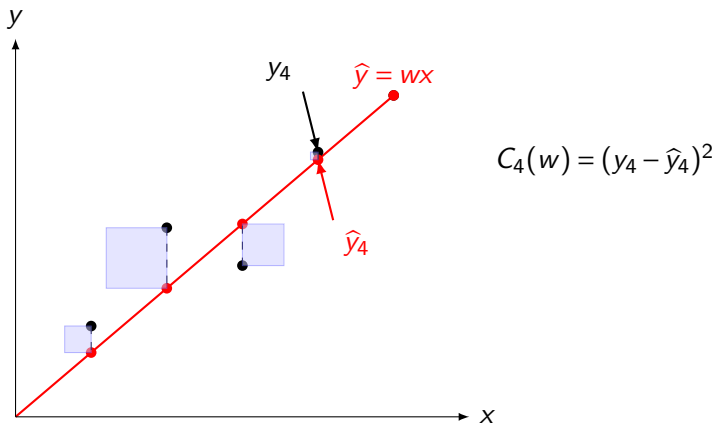
$$C_2(w) = (y_2 - \hat{y}_2)^2$$

Find the best lines, i.e choose a suitable value for  $w$

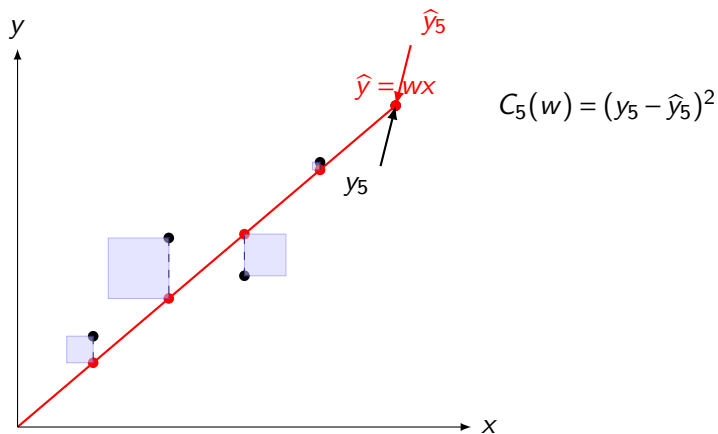


$$C_3(w) = (y_3 - \hat{y}_3)^2$$

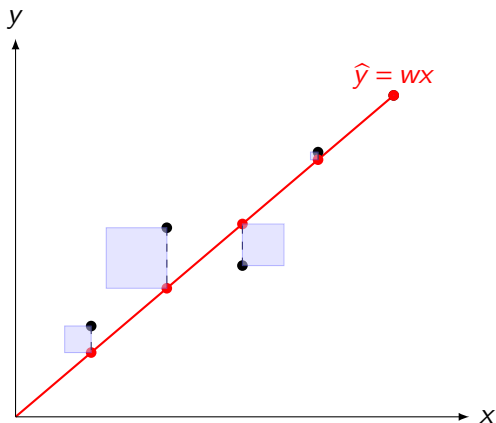
Find the best lines, i.e choose a suitable value for  $w$



Find the best lines, i.e choose a suitable value for  $w$



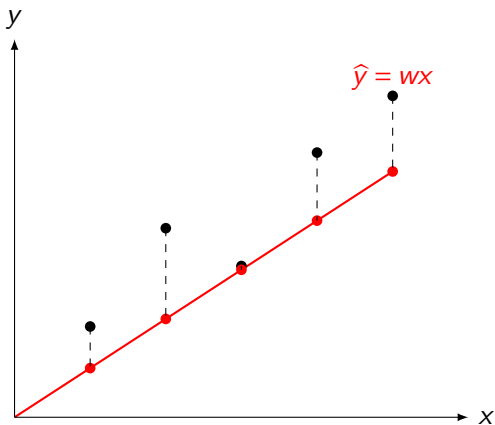
Find the best lines, i.e choose a suitable value for  $w$



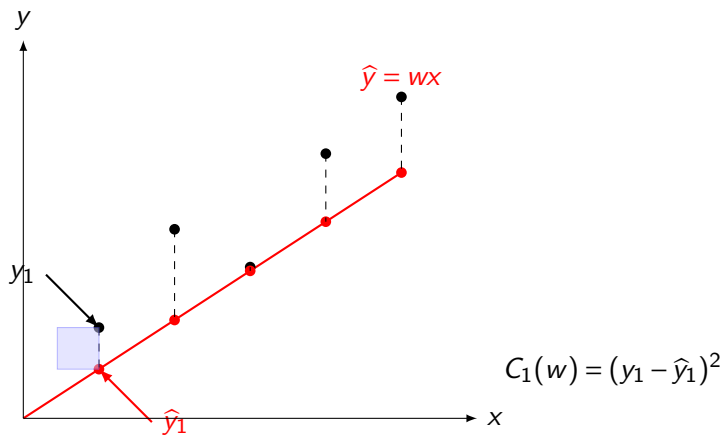
Here  $w = 0.85$  and total loss is:

$$C(w) = \sum_{i=1}^5 C_i(w) = 5.4$$

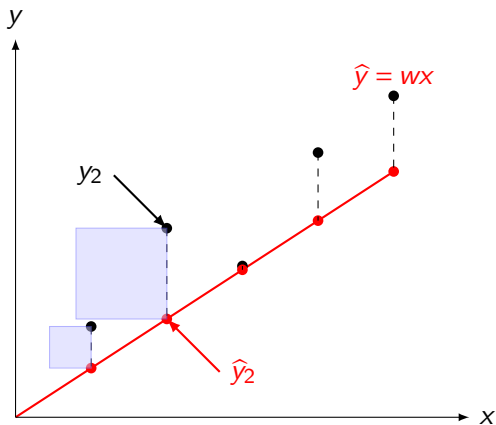
Another try with  $w = 0.65$



Another try with  $w = 0.65$

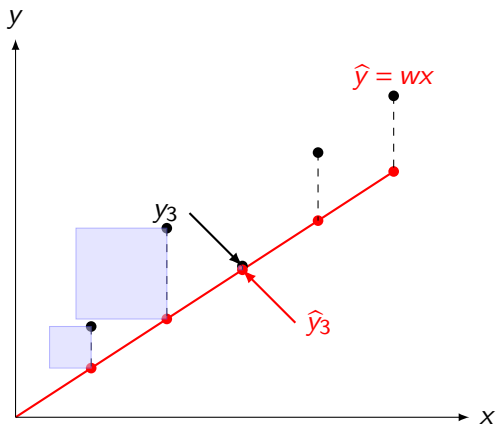


Another try with  $w = 0.65$



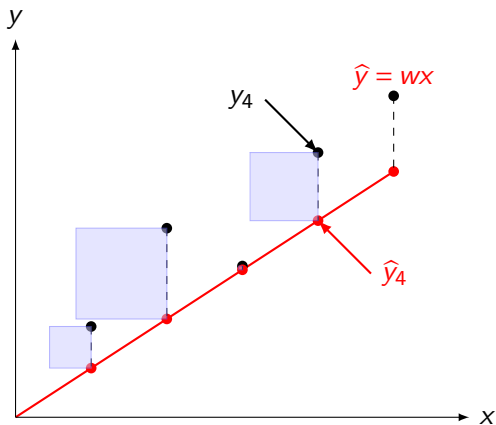
$$C_2(w) = (y_2 - \hat{y}_2)^2$$

Another try with  $w = 0.65$



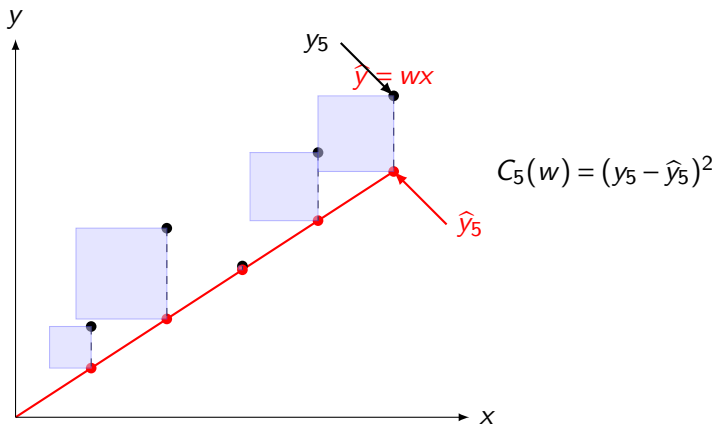
$$C_3(w) = (y_3 - \hat{y}_3)^2$$

Another try with  $w = 0.65$

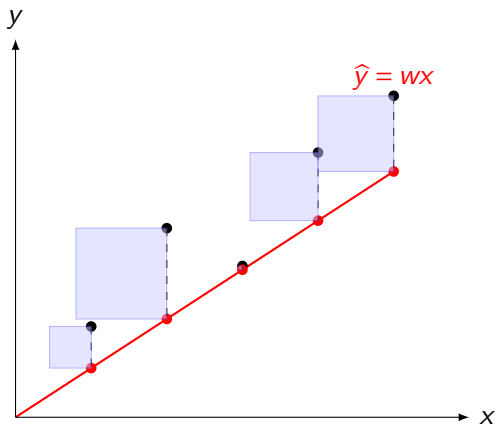


$$C_4(w) = (y_4 - \hat{y}_4)^2$$

Another try with  $w = 0.65$



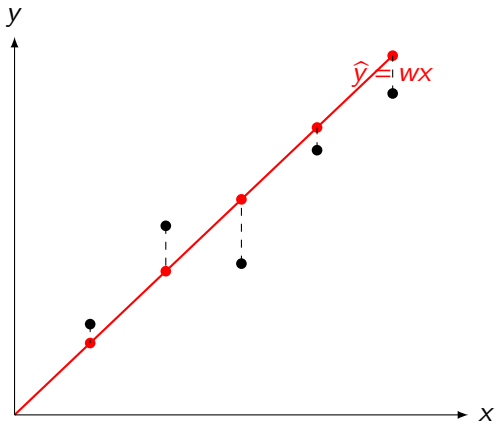
Another try with  $w = 0.65$



$$C(w) = \sum_{i=1}^5 C_i(w) = 12.3$$

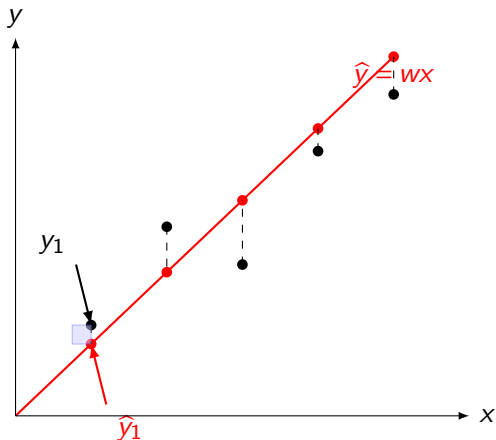
## Another try with $w = 0.95$

Here a **linear activation function** ( $\sigma(u) = u$ )



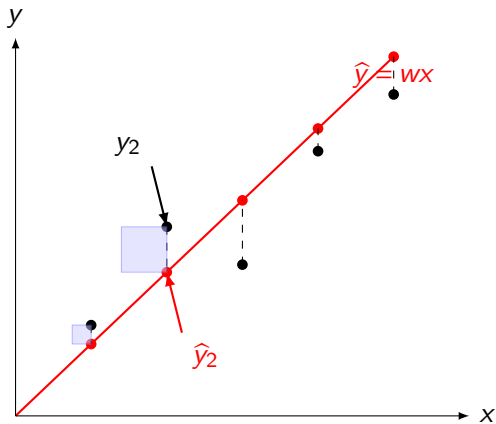
## Another try with $w = 0.95$

Here a **linear activation function** ( $\sigma(u) = u$ )



## Another try with $w = 0.95$

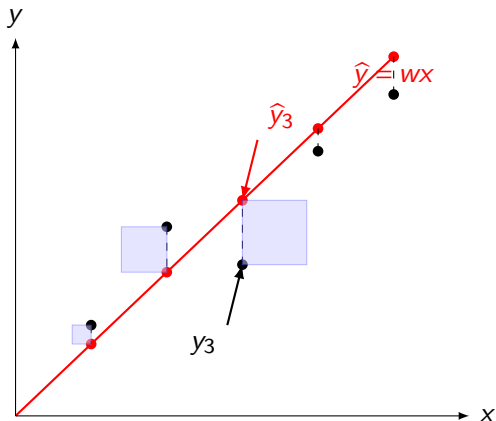
Here a **linear activation function** ( $\sigma(u) = u$ )



$$C_2(w) = (y_2 - \hat{y}_2)^2$$

## Another try with $w = 0.95$

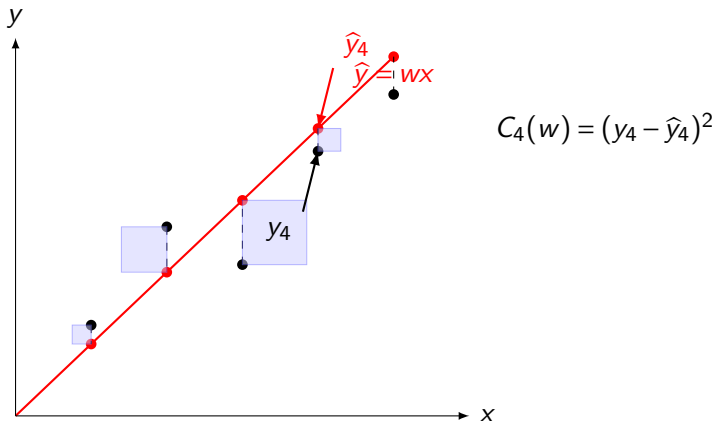
Here a **linear activation function** ( $\sigma(u) = u$ )



$$C_3(w) = (y_3 - \hat{y}_3)^2$$

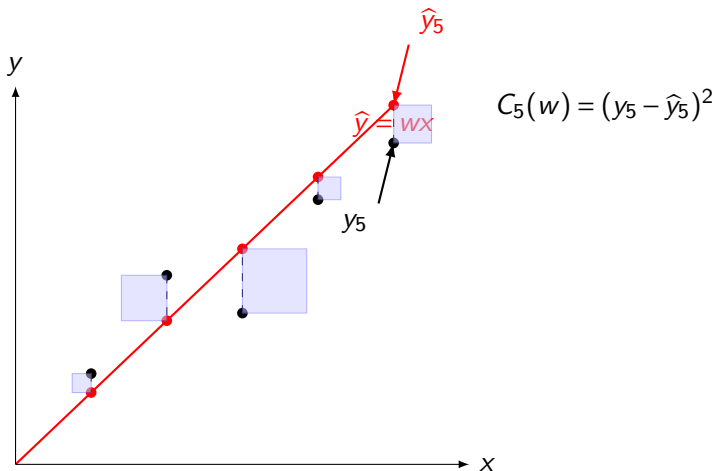
## Another try with $w = 0.95$

Here a **linear activation function** ( $\sigma(u) = u$ )



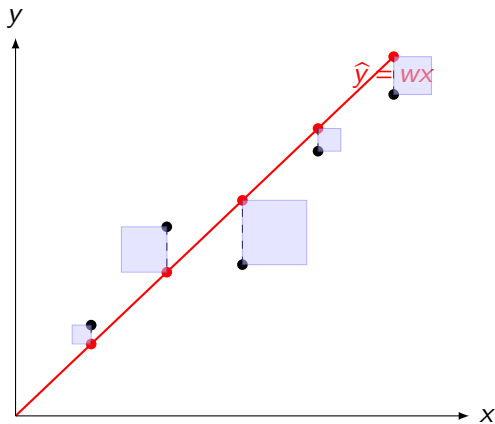
## Another try with $w = 0.95$

Here a **linear activation function** ( $\sigma(u) = u$ )



## Another try with $w = 0.95$

Here a **linear activation function** ( $\sigma(u) = u$ )



$$C(w) = \sum_{i=1}^5 C_i(w) = 4.9$$

# We want to minimize the loss function

- ▶ DL exploits iterative optimisation method to **train** (find) suitable parameter.
- ▶ Gradient Descent (GD) algorithm and variations are the key tools.
- ▶ **Gradient** means derivative (slope of the function) for one parameter model.
- ▶ An estimate of the slope of the function (rise divided by run) at a specific point  $\theta$ :

$$\begin{aligned}\frac{\text{Rise}}{\text{Run}} &= \frac{C(\theta + \Delta) - C(\theta)}{(\theta + \Delta) - \theta} \\ &\approx C'(\theta) \quad \text{when } \Delta \text{ positive small number}\end{aligned}$$

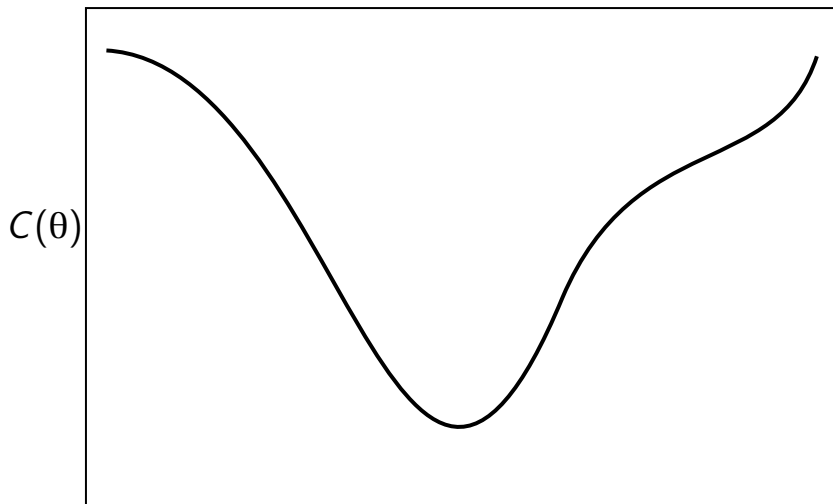
# We want to minimize the loss function

- ▶ DL exploits iterative optimisation method to **train** (find) suitable parameter.
- ▶ Gradient Descent (GD) algorithm and variations are the key tools.
- ▶ **Gradient** means derivative (slope of the function) for one parameter model.
- ▶ An estimate of the slope of the function (rise divided by run) at a specific point  $\theta$ :

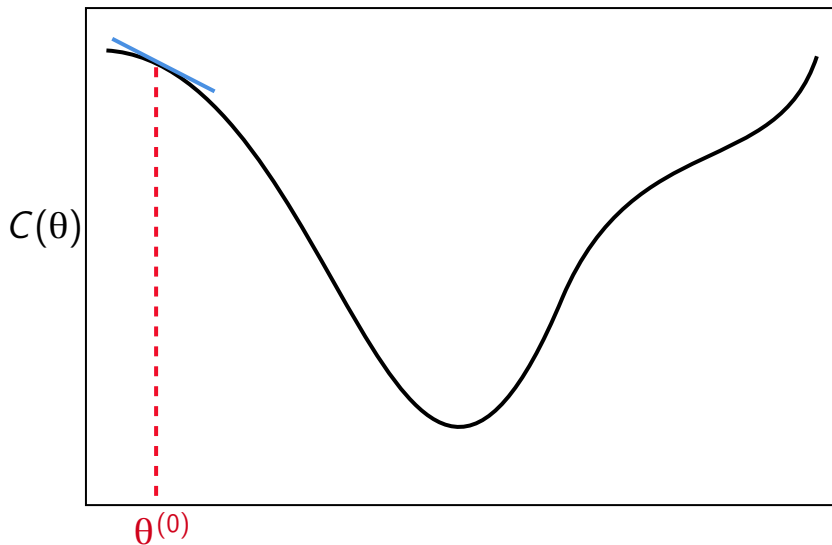
$$\begin{aligned}\frac{\text{Rise}}{\text{Run}} &= \frac{C(\theta + \Delta) - C(\theta)}{(\theta + \Delta) - \theta} \\ &\approx C'(\theta) \quad \text{when } \Delta \text{ positive small number}\end{aligned}$$

- ▶ Gradient is related to vector parameter: noted  $\nabla C(\theta)$  ( $\theta \in \mathbb{R}^p$ )

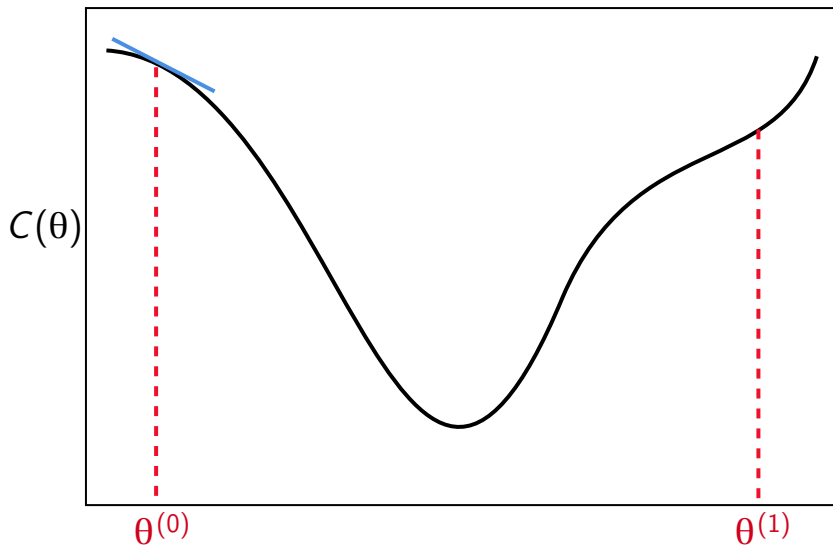
Gradient Descent in action:  $\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla C(\theta^{(t)})$



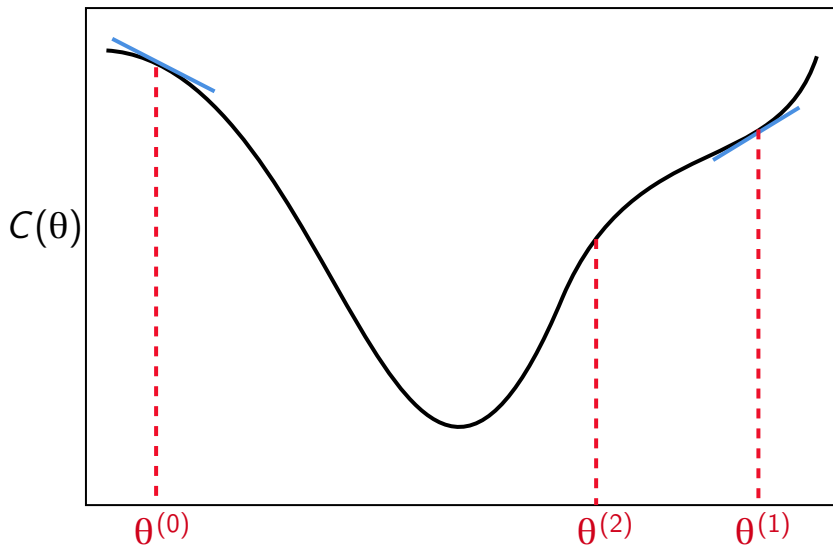
Gradient Descent: Initial guess  $\theta^{(0)}$  and compute  $\nabla C(\theta^{(0)})$



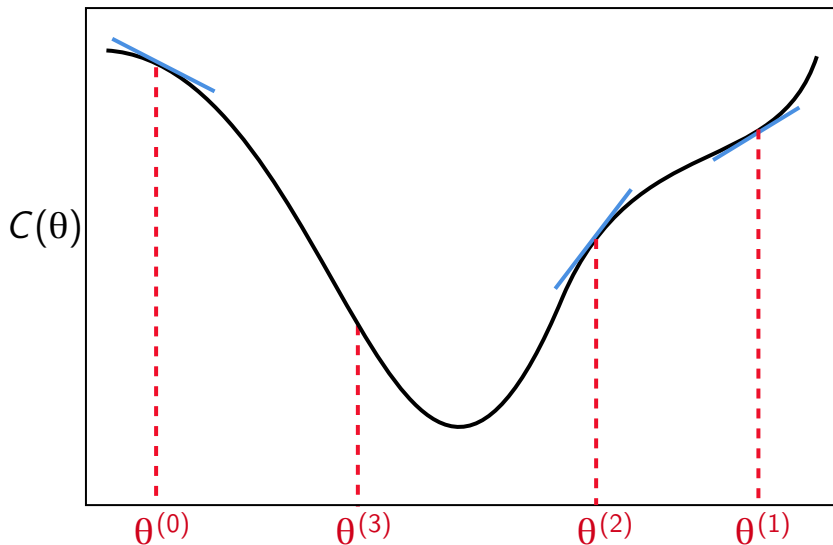
Gradient Descent moves:  $\theta^{(1)} = \theta^{(0)} - \alpha \nabla C(\theta^{(0)})$



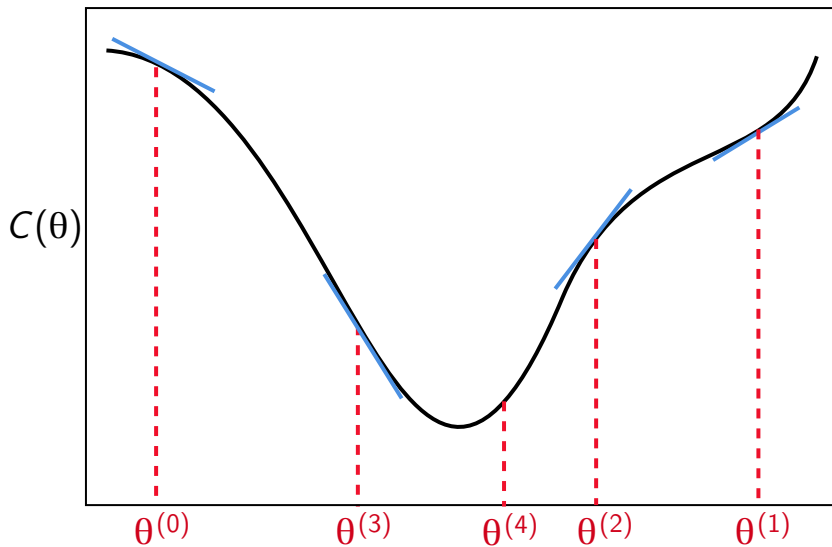
Gradient Descent moves:  $\theta^{(2)} = \theta^{(1)} - \alpha \nabla C(\theta^{(1)})$



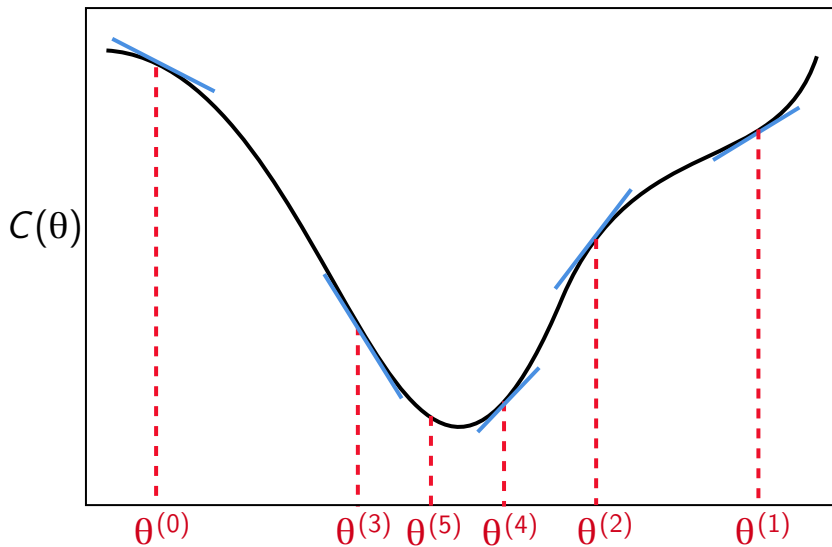
Gradient Descent moves:  $\theta^{(3)} = \theta^{(2)} - \alpha \nabla C(\theta^{(2)})$



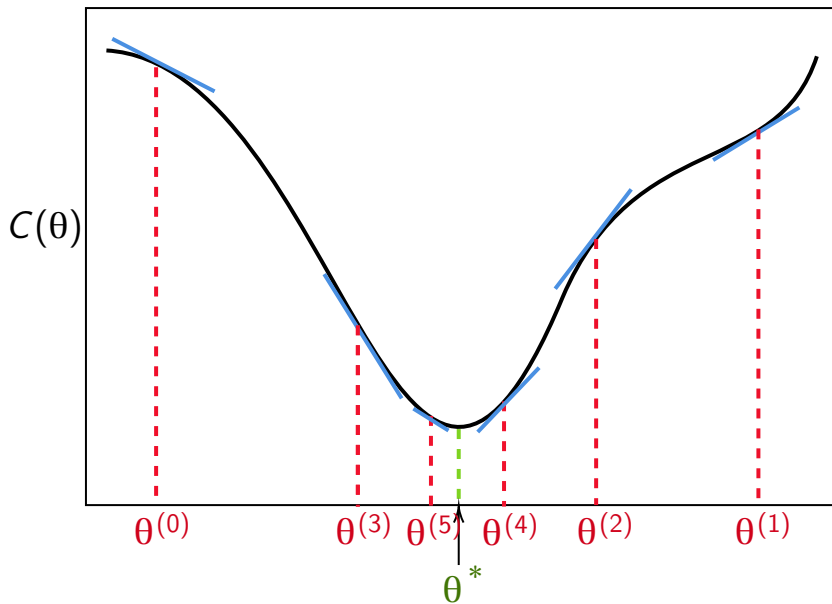
Gradient Descent moves:  $\theta^{(4)} = \theta^{(3)} - \alpha \nabla C(\theta^{(3)})$



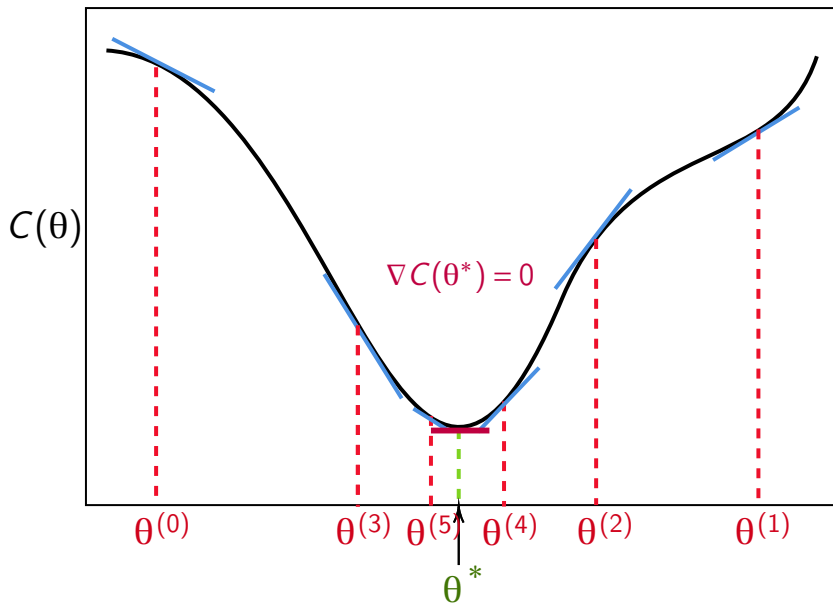
Gradient Descent moves:  $\theta^{(5)} = \theta^{(4)} - \alpha \nabla C(\theta^{(4)})$



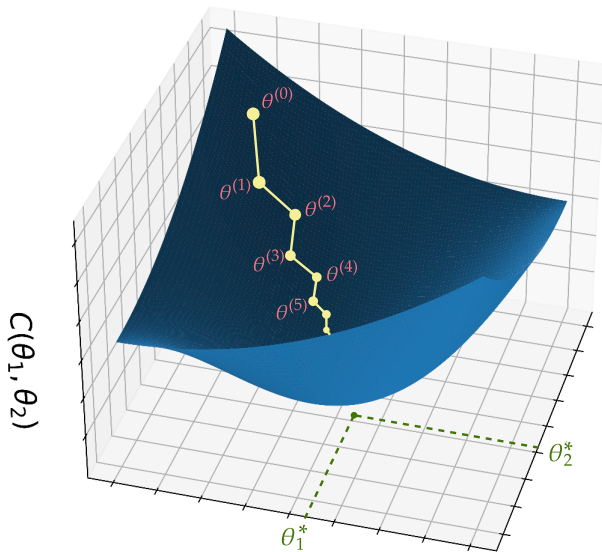
Gradient Descent moves:  $\theta^* = \theta^{(5)} - \alpha \nabla C(\theta^{(5)})$



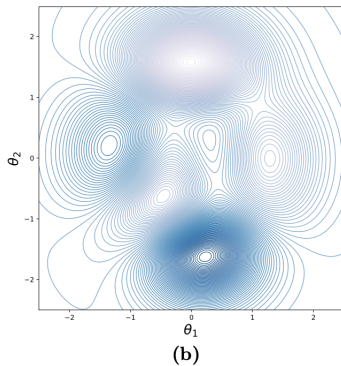
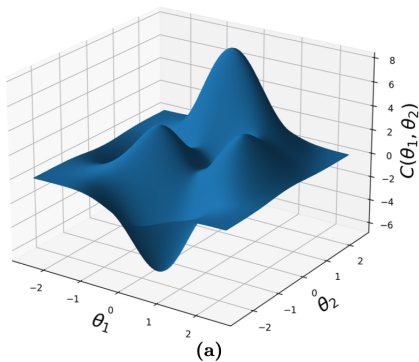
Gradient Descent terminates:  $\nabla C(\theta^*) \approx 0$



## Loss landscape: two parameters

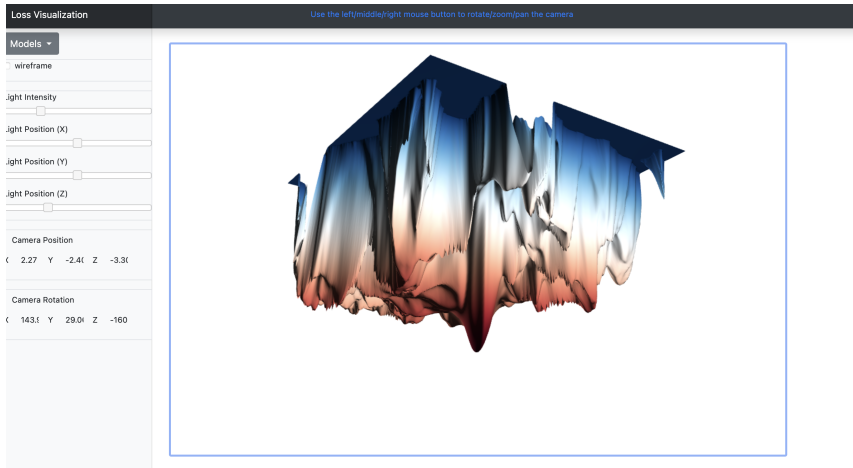


# Loss landscape: 3D surface plot and topographical map



# Loss landscape is quite complex

## Interactive loss landscape visualization



## Why Gradient Descent is expensive?

$$C(\theta; \text{Data}) = \frac{1}{n} \sum_{i=1}^n C_i(\theta), \quad \text{i.e.,} \quad C_i(\theta) = (y_i - \hat{y}_i)^2$$

## Why Gradient Descent is expensive?

$$C(\theta; \text{Data}) = \frac{1}{n} \sum_{i=1}^n C_i(\theta), \quad \text{i.e.,} \quad C_i(\theta) = (y_i - \hat{y}_i)^2$$

- ▶ To compute the gradient, GD must look at **all training data** at every step:

$$\nabla C(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla C_i(\theta)$$

- ▶ For modern datasets: **millions of data points** and **millions of parameters**.
- ▶ Each update becomes **slow** and **computationally expensive**.

**Key idea:** *Deep learning replaces full Gradient Descent with faster approximations, such as **Stochastic Gradient Descent (SGD)** and **mini-batch GD**, which use only a small subset of the data at each step.*

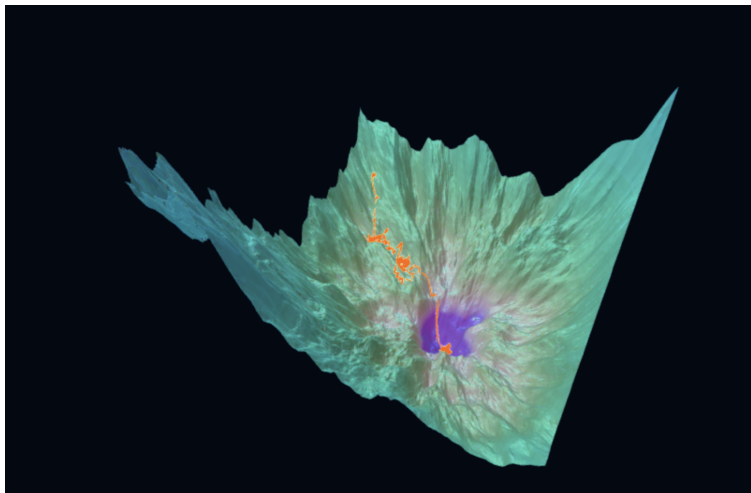
## Question 5

*Does gradient descent guarantee that we find the global minimum of the loss function?*



Visualisation: <https://losslandscape.com/>

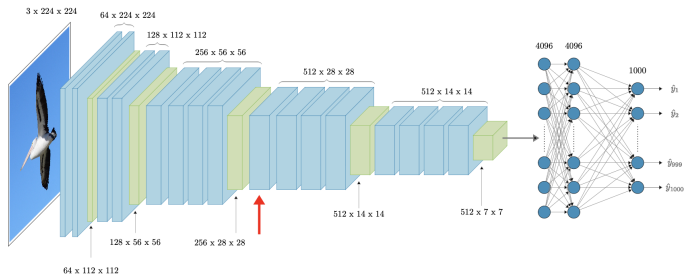
Explorer



# Visualisation: video

<https://losslandscape.com/videos/>

# A Real Convolutional Network: VGG19



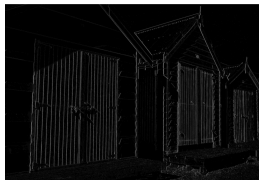
- ▶ Input: a colour image,  $3 \times 224 \times 224$ , the 3 being the red, green and blue channels
- ▶ Output: a vector of 1,000 probabilities, one per ImageNet class
- ▶ In between: a long stack of **convolutions**, and about **144 million** parameters

*So what does a single one of those layers actually do?*

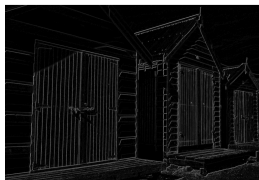
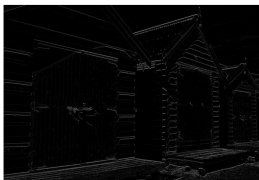
# Filtering: an Old Idea from Signal Processing



(a)



(b)



(a) the image    (b) vertical edges    (c) horizontal edges    (d) the two added

*Filtering removes or accentuates a particular kind of information. The operation behind it is the **convolution**.*

# The Filter Is Just a Small Matrix

The images above were produced by the **Sobel filters**, two matrices of nine numbers:

horizontal edges

$$W = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

vertical edges

$$W = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

# The Filter Is Just a Small Matrix

The images above were produced by the **Sobel filters**, two matrices of nine numbers:

horizontal edges

$$W = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

vertical edges

$$W = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

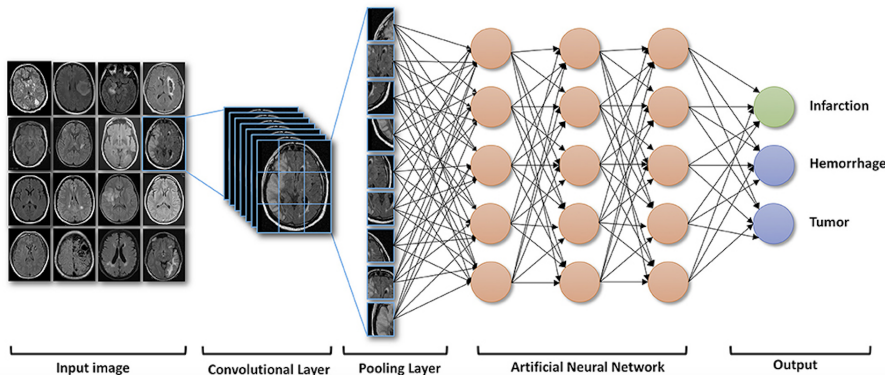
In classical signal processing, these nine numbers are **designed** by an engineer who already knows what to look for.

In a convolutional network they are **learned**, exactly like the weights of any other layer. The network decides by itself what is worth detecting.

*A filter is a weight matrix. Learning a filter means learning its nine numbers.*

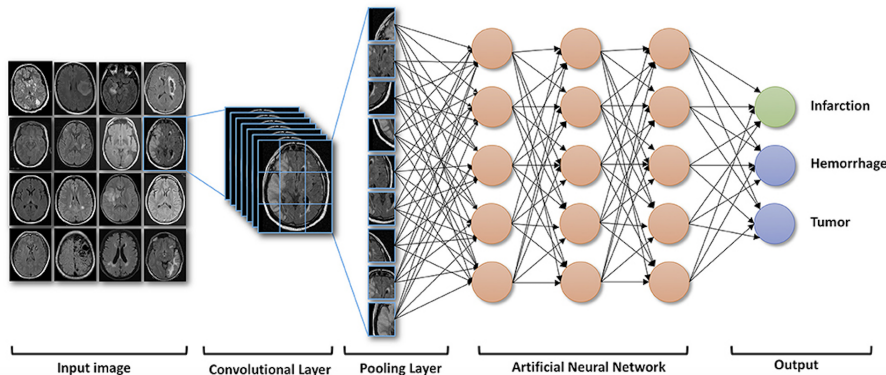
# CNN: the math behind ?

## B Computer Neural Network(Convolutional Neural Network)



# CNN: the math behind ?

## B Computer Neural Network(Convolutional Neural Network)



The key is the **Convolution operation**

# Convolution 1D

Given an **input vector**  $x = [x_1, x_2, \dots, x_{10}]$  and a **kernel (filter)**  $w = [w_1, w_2, w_3]$ , the output  $z$  of the 1D convolution operation is :

$$Z = W \star X$$

where  $\star$  denotes the convolution operation. Each element is computed as:

$$Z_i = w_1 \cdot x_i + w_2 \cdot x_{i+1} + w_3 \cdot x_{i+2}$$

|        |      |     |     |   |   |   |   |   |   |    |
|--------|------|-----|-----|---|---|---|---|---|---|----|
| Input  | 1    | 2   | 3   | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| Kernel | 0.3  | 0.5 | 0.1 |   |   |   |   |   |   |    |
| Output | 1.60 |     |     |   |   |   |   |   |   |    |

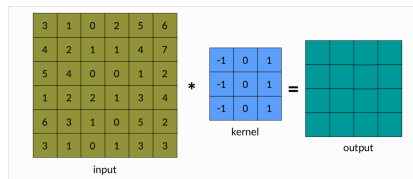
$$1 \times 0.3 + 2 \times 0.5 + 3 \times 0.1 = 1.60$$



# Convolution 2D: $z = W \star x$

A  $3 \times 3$  Kernel  $W$  applied on an  $6 \times 6$  input matrix

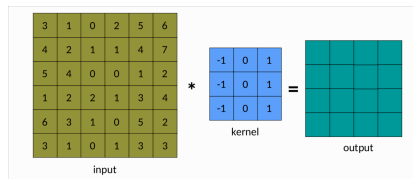
$$\underbrace{\begin{bmatrix} -11 & -4 & 9 & 12 \\ -7 & -6 & 5 & 11 \\ -9 & -8 & 6 & 7 \\ -7 & -4 & 8 & 7 \end{bmatrix}}_z = \underbrace{\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}}_W \star \underbrace{\begin{bmatrix} 3 & 1 & 0 & 2 & 5 & 6 \\ 4 & 2 & 1 & 1 & 4 & 7 \\ 5 & 4 & 0 & 0 & 1 & 2 \\ 1 & 2 & 2 & 1 & 3 & 4 \\ 6 & 3 & 1 & 0 & 5 & 2 \\ 3 & 1 & 0 & 1 & 3 & 3 \end{bmatrix}}_x$$



## Convolution 2D: $z = W \star x$

A  $3 \times 3$  Kernel  $W$  applied on an  $6 \times 6$  input matrix

$$\underbrace{\begin{bmatrix} -11 & -4 & 9 & 12 \\ -7 & -6 & 5 & 11 \\ -9 & -8 & 6 & 7 \\ -7 & -4 & 8 & 7 \end{bmatrix}}_z = \underbrace{\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}}_W \star \underbrace{\begin{bmatrix} 3 & 1 & 0 & 2 & 5 & 6 \\ 4 & 2 & 1 & 1 & 4 & 7 \\ 5 & 4 & 0 & 0 & 1 & 2 \\ 1 & 2 & 2 & 1 & 3 & 4 \\ 6 & 3 & 1 & 0 & 5 & 2 \\ 3 & 1 & 0 & 1 & 3 & 3 \end{bmatrix}}_x$$



$$z_{i,j} = \sum_{m=0}^2 \sum_{n=0}^2 W_{m,n} \cdot x_{i+m-1,j+n-1}$$

# Example of 2d convolution filter

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |



Identity Kernel: Just the starting image itself.

$\frac{1}{9}$

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |



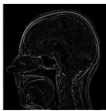
Box Blur Kernel: Each pixel is averaged equally with its 8 neighbors.

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |



Sharpen Kernel: Differences emphasized with its adjacent pixel values.

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |



Outline ("Edge") Kernel: Highlights neighboring pixels with large differences of intensity.

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |



Left-to-right Sobel Kernel: Used to accentuate differences between pixels along the horizontal axis

|    |    |    |
|----|----|----|
| -1 | -2 | -1 |
| 0  | 0  | 0  |
| 1  | 2  | 1  |



Superior-to-inferior Sobel Kernel: Used to accentuate differences between pixels along the vertical axis.

# Example of 2d convolution filter

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |



Identity Kernel: Just the starting image itself.

$\frac{1}{9}$

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |



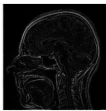
Box Blur Kernel: Each pixel is averaged equally with its 8 neighbors.

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 5  | -1 |
| 0  | -1 | 0  |



Sharpen Kernel: Differences emphasized with its adjacent pixel values.

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |



Outline ("Edge") Kernel: Highlights neighboring pixels with large differences of intensity.

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |



Left-to-right Sobel Kernel: Used to accentuate differences between pixels along the horizontal axis

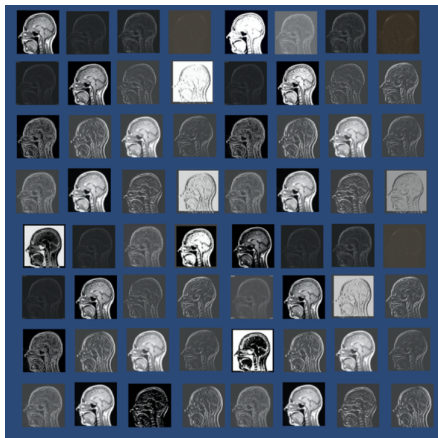
|    |    |    |
|----|----|----|
| -1 | -2 | -1 |
| 0  | 0  | 0  |
| 1  | 2  | 1  |



Superior-to-inferior Sobel Kernel: Used to accentuate differences between pixels along the vertical axis.

The key is the **Convolution operation**

# CNN: Kernels are learned



# Why Convolutional Kernels?

**Key idea:** Convolutional kernels allow neural networks to efficiently detect **local patterns** that can appear **anywhere** in the input.

## ▶ Locality

- ▶ Nearby pixels (or signal values) are strongly related
- ▶ Clinically relevant structures (edges, textures, lesions) are local
- ▶ Convolution focuses on small neighborhoods

## ▶ Translation invariance

- ▶ A relevant pattern matters regardless of its position
- ▶ The same kernel is applied across the entire image
- ▶ Enables detection of the same feature anywhere

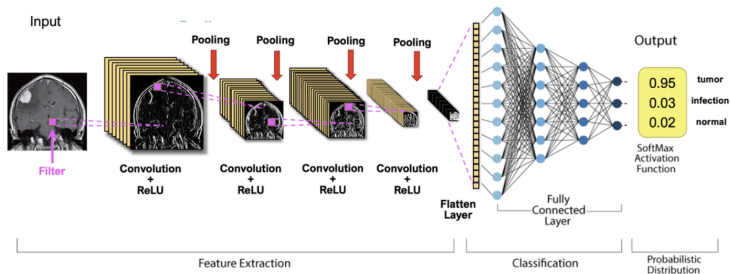
## ▶ Parameter efficiency

- ▶ Fully connected layers require many parameters
- ▶ Convolution uses shared weights
- ▶ Fewer parameters  $\Rightarrow$  better generalization

**Take-home message:** *What matters locally matters everywhere.*

# CNN as a Hierarchical Feature Extractor

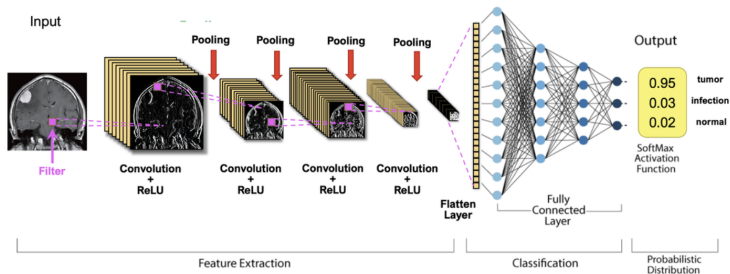
A CNN is a sequence of convolutional layers that progressively extract more abstract features before making a prediction.



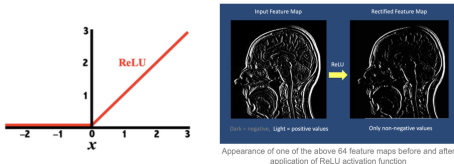
**Input** → **Conv + ReLU** → **Conv + ReLU** → **Conv + ReLU**  
Image / Signal   Edges, textures   Shapes, regions   High-level patterns

# CNN as a Hierarchical Feature Extractor

A CNN is a sequence of convolutional layers that progressively extract more abstract features before making a prediction.

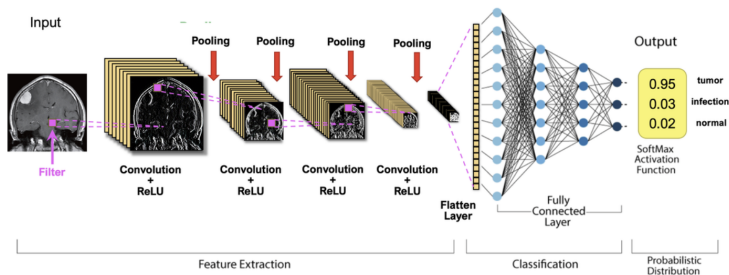


Input → Conv + ReLU → Conv + ReLU → Conv + ReLU  
Image / Signal   Edges, textures   Shapes, regions   High-level patterns



# CNN as a Hierarchical Feature Extractor

A CNN is a sequence of convolutional layers that progressively extract more abstract features before making a prediction.



Input → Conv + ReLU → Conv + ReLU → Conv + ReLU  
Image / Signal   Edges, textures   Shapes, regions   High-level patterns

- ▶ **Convolution layers** perform **feature extraction**
- ▶ **Fully connected layers** perform **decision making**

*Analogy: from edges → structures → diagnosis*

## Question 6?

*Do convolutional neural networks reduce the number of parameters by sharing the same filter across different parts of the image?*

# From Words to Numbers

A neural network only knows how to **multiply and add numbers**.

But a clinical note is a sequence of *symbols*:

*"The patient reports a headache and took ibuprofen."*

# From Words to Numbers

A neural network only knows how to **multiply and add numbers**.

But a clinical note is a sequence of *symbols*:

*“The patient reports a headache and took ibuprofen.”*

Two questions to solve before any deep learning

1. How do we cut the text into elementary units? → **tokens**
2. How do we turn each unit into a vector? → **word embedding**

*This is the hidden first layer of every LLM you use.*

## Step 1: Tokenization

A **token** is the elementary unit of text seen by the model; here, one token = one word.

The patient reports a headache and took ibuprofen .

# Step 1: Tokenization

A **token** is the elementary unit of text seen by the model; here, one token = one word.

The patient reports a headache and took ibuprofen .

- ▶ The list of all possible tokens is the **vocabulary**, of size  $V$  (typically  $V \approx 100,000$ )
- ▶ Each token is then replaced by its **index** in the vocabulary
- ▶ From now on the model never sees letters: only a **sequence of numbers**

# Step 1: Tokenization

A **token** is the elementary unit of text seen by the model; here, one token = one word.

The patient reports a headache and took ibuprofen .

- ▶ The list of all possible tokens is the **vocabulary**, of size  $V$  (typically  $V \approx 100,000$ )
- ▶ Each token is then replaced by its **index** in the vocabulary
- ▶ From now on the model never sees letters: only a **sequence of numbers**

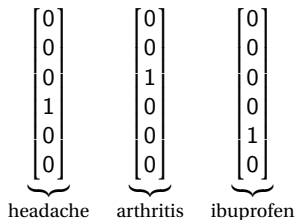
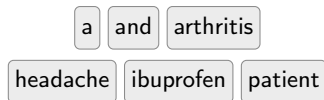
*Note: with whole words, anything absent from the list (a new drug name, a proper noun, a typo) becomes  $\langle \text{UNK} \rangle$  and is lost. That was the RNN/LSTM situation. Today's LLMs avoid it by cutting rare words into pieces (ibuprofen  $\rightarrow$  ib | upro | fen): a word never seen before can still be written, and its pieces still carry information.*

## Step 2, First Attempt: One-Hot Encoding

One 1 at the position of the token, 0 everywhere else:

$$\text{token } i \mapsto \mathbf{e}_i \in \mathbb{R}^V$$

Toy vocabulary,  $V = 6$ :



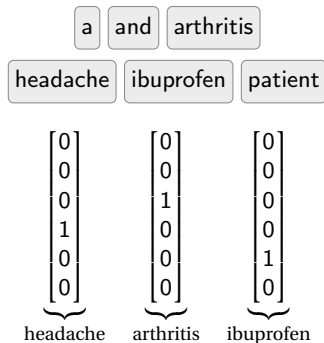
*Nothing new: this is the **dummy coding** of a categorical variable, with one category per word.*

## Step 2, First Attempt: One-Hot Encoding

One 1 at the position of the token, 0 everywhere else:

$$\text{token } i \mapsto e_i \in \mathbb{R}^V$$

Toy vocabulary,  $V = 6$ :



Nothing new: this is the **dummy coding** of a categorical variable, with one category per word.

### Why it does not work

1. **Too large:**  $V \approx 100,000$  numbers for a single word
2. **Too sparse:** one useful entry, the rest is zeros

3. **No meaning:**

$$e_i^\top e_j = 0, \quad \|e_i - e_j\| = \sqrt{2}$$

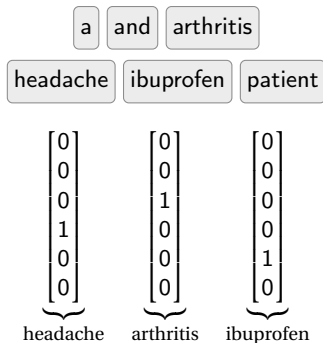
all words are orthogonal and equidistant: *headache* is exactly as far from *migraine* as from *bicycle*

## Step 2, First Attempt: One-Hot Encoding

One 1 at the position of the token, 0 everywhere else:

$$\text{token } i \mapsto \mathbf{e}_i \in \mathbb{R}^V$$

Toy vocabulary,  $V = 6$ :



*Nothing new: this is the **dummy coding** of a categorical variable, with one category per word.*

### Why it does not work

1. **Too large:**  $V \approx 100,000$  numbers for a single word
2. **Too sparse:** one useful entry, the rest is zeros

3. **No meaning:**

$$\mathbf{e}_i^\top \mathbf{e}_j = 0, \quad \|\mathbf{e}_i - \mathbf{e}_j\| = \sqrt{2}$$

all words are orthogonal and equidistant: *headache* is exactly as far from *migraine* as from *bicycle*

Using the **dot product** to compare words, one-hot encoding returns 0 for every comparison.

## Step 2: What Is a Word Embedding?

Give each word a handful of **coordinates**, instead of one spike in a huge space:

$$\text{word} \mapsto \mathbf{x} \in \mathbb{R}^d, \quad d \approx 300 \lll V \approx 100,000$$

## Step 2: What Is a Word Embedding?

Give each word a handful of **coordinates**, instead of one spike in a huge space:

$$\text{word} \mapsto \mathbf{x} \in \mathbb{R}^d, \quad d \approx 300 \lll V \approx 100,000$$

|          | $d$ coordinates |       |       |       |
|----------|-----------------|-------|-------|-------|
| a        | 0.02            | -0.11 | 0.07  | 0.01  |
| and      | -0.05           | 0.03  | 0.09  | -0.02 |
| headache | 0.79            | 0.21  | -0.39 | 0.71  |
| migraine | 0.81            | 0.12  | -0.44 | 0.63  |
| bicycle  | -0.22           | 0.88  | 0.41  | -0.05 |
| patient  | 0.14            | -0.30 | 0.55  | 0.22  |

## Step 2: What Is a Word Embedding?

Give each word a handful of **coordinates**, instead of one spike in a huge space:

$$\text{word} \mapsto \mathbf{x} \in \mathbb{R}^d, \quad d \approx 300 \lll V \approx 100,000$$

|          | $d$ coordinates |       |       |       |
|----------|-----------------|-------|-------|-------|
| a        | 0.02            | -0.11 | 0.07  | 0.01  |
| and      | -0.05           | 0.03  | 0.09  | -0.02 |
| headache | 0.79            | 0.21  | -0.39 | 0.71  |
| migraine | 0.81            | 0.12  | -0.44 | 0.63  |
| bicycle  | -0.22           | 0.88  | 0.41  | -0.05 |
| patient  | 0.14            | -0.30 | 0.55  | 0.22  |

- ▶ A simple **dictionary**: one line per word,  $d$  numbers per line
- ▶ *headache* and *migraine* now have **similar lines**; *bicycle* does not
- ▶ Short and dense: no more 100,000 zeros

## Step 2: What Is a Word Embedding?

Give each word a handful of **coordinates**, instead of one spike in a huge space:

$$\text{word} \mapsto \mathbf{x} \in \mathbb{R}^d, \quad d \approx 300 \lll V \approx 100,000$$

|          | $d$ coordinates |       |       |       |
|----------|-----------------|-------|-------|-------|
| a        | 0.02            | -0.11 | 0.07  | 0.01  |
| and      | -0.05           | 0.03  | 0.09  | -0.02 |
| headache | 0.79            | 0.21  | -0.39 | 0.71  |
| migraine | 0.81            | 0.12  | -0.44 | 0.63  |
| bicycle  | -0.22           | 0.88  | 0.41  | -0.05 |
| patient  | 0.14            | -0.30 | 0.55  | 0.22  |

- ▶ A simple **dictionary**: one line per word,  $d$  numbers per line
- ▶ *headache* and *migraine* now have **similar lines**; *bicycle* does not
- ▶ Short and dense: no more 100,000 zeros

But who writes these numbers?

*Certainly not a human being: there are  $100,000 \times 300$  of them.*

# Where Do These Numbers Come From?

The only assumption needed

*“You shall know a word by the company it keeps”* (Firth, 1957)

# Where Do These Numbers Come From?

## The only assumption needed

*"You shall know a word by the company it keeps"* (Firth, 1957)

Hide a word in a sentence and ask a small network to guess it from its neighbours:

The patient reports a ? and took ibuprofen.

*headache?*    *migraine?*    *fever?*    but certainly not *bicycle*.

# Where Do These Numbers Come From?

## The only assumption needed

*"You shall know a word by the company it keeps"* (Firth, 1957)

Hide a word in a sentence and ask a small network to guess it from its neighbours:

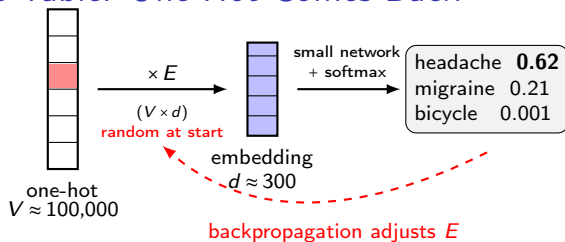
The patient reports a ? and took ibuprofen.

*headache?*    *migraine?*    *fever?*    but certainly not *bicycle*.

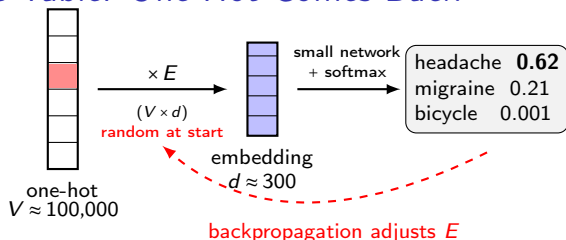
## Consequence

To succeed, the network is **forced** to give close coordinates to words that fill the same holes: the geometry appears **by itself**, with no dictionary and no expert annotation, from raw text alone.

# Building the Table: One-Hot Comes Back

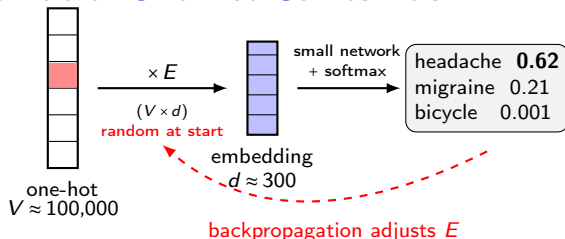


# Building the Table: One-Hot Comes Back



- ▶ The input is the **one-hot** vector, all we have before any embedding exists
- ▶  $E$  starts **at random**; multiplying a one-hot by  $E$  picks row  $i$
- ▶ **GloVe** (2014) takes the other route: **count** how often two words occur close together (*headache* next to *patient* far more often than next to *bicycle*), then find the  $d$  coordinates that reproduce these  $V \times V$  counts: a **matrix factorisation**, with no network

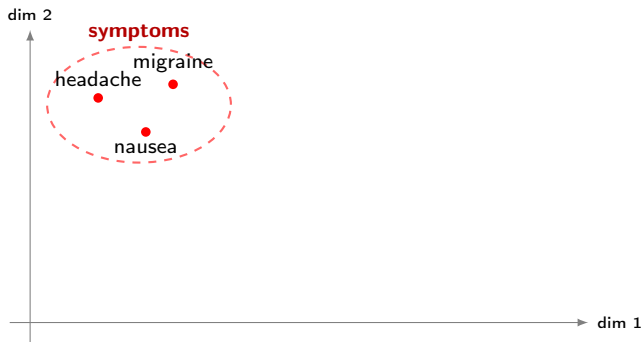
# Building the Table: One-Hot Comes Back



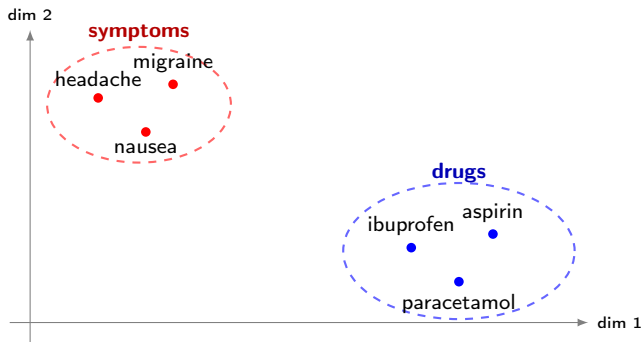
- ▶ The input is the **one-hot** vector, all we have before any embedding exists
- ▶  $E$  starts **at random**; multiplying a one-hot by  $E$  picks row  $i$
- ▶ **GloVe** (2014) takes the other route: **count** how often two words occur close together (*headache* next to *patient* far more often than next to *bicycle*), then find the  $d$  coordinates that reproduce these  $V \times V$  counts: a **matrix factorisation**, with no network

$E$  is adjusted so that the softmax favours the *true* missing word. The network is then thrown away and only  $E$  is kept: the embeddings are the **by-product** of the task (**word2vec**, 2013).

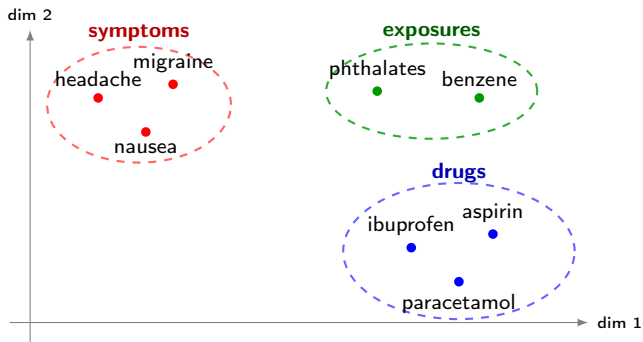
# The Embedding Space



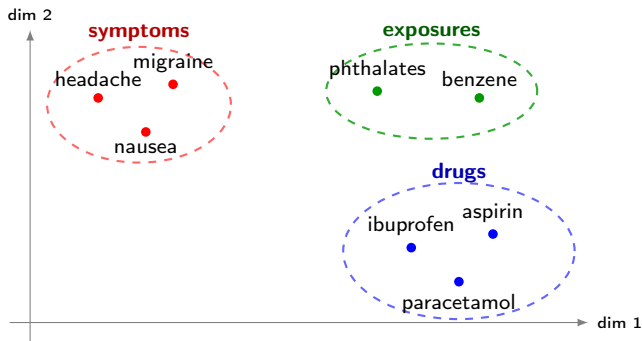
# The Embedding Space



# The Embedding Space



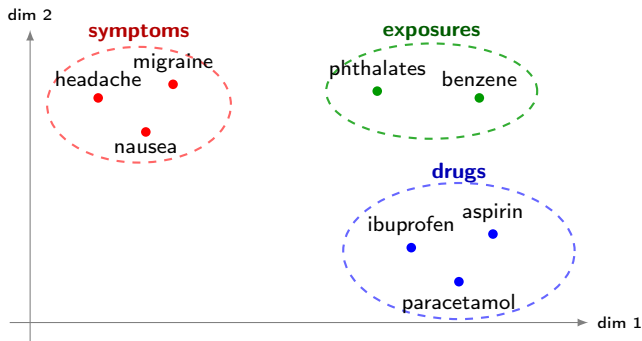
# The Embedding Space



Closeness between two words is measured by the **dot product**:

$$\cos(\theta_{ij}) = \frac{x_i^T x_j}{\|x_i\| \|x_j\|}$$

# The Embedding Space

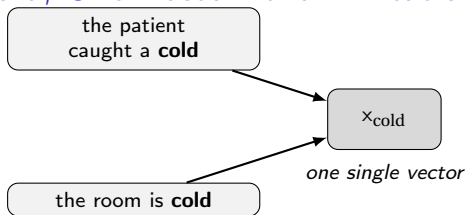


Closeness between two words is measured by the **dot product**:

$$\cos(\theta_{ij}) = \frac{x_i^T x_j}{\|x_i\| \|x_j\|}$$

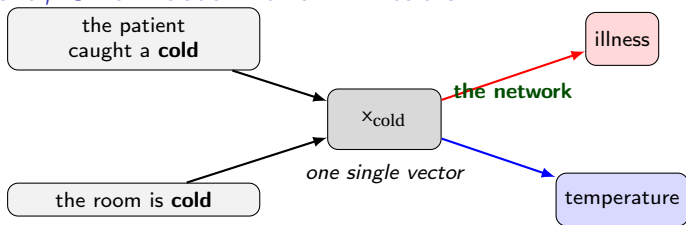
*Words now live in a space where the **dot product means something**:  
meaning has become geometry.*

# One Word, One Vector: the Limitation



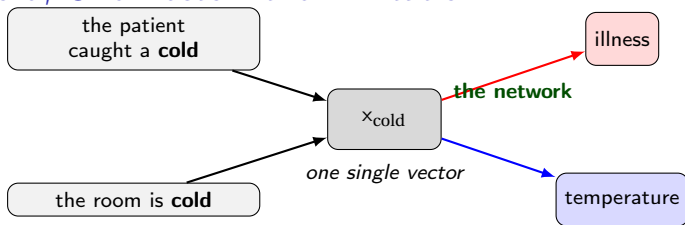
- ▶ With **word2vec** or **GloVe**, *cold* has *one* vector for ever: a blurred average of its meanings
- ▶ Any embedding table does the same, whatever the model that follows: the **same** starting vector in both sentences

# One Word, One Vector: the Limitation



- ▶ With **word2vec** or **GloVe**, *cold* has *one* vector for ever: a blurred average of its meanings
- ▶ Any embedding table does the same, whatever the model that follows: the **same** starting vector in both sentences
- ▶ Telling them apart is therefore the job of the **network that reads the sentence**: it has to **reshape each vector using its neighbours**

# One Word, One Vector: the Limitation



- ▶ With **word2vec** or **GloVe**, *cold* has *one* vector for ever: a blurred average of its meanings
- ▶ Any embedding table does the same, whatever the model that follows: the **same** starting vector in both sentences
- ▶ Telling them apart is therefore the job of the **network that reads the sentence**: it has to **reshape each vector using its neighbours**

Everything that follows answers one question: **how does a model use the neighbours to reshape a word?**

# Sequences: a Different Kind of Data

A **sequence** is a list of observations whose **order carries information**: a clinical note, an ECG, a series of exposure measurements, a DNA strand.

# Sequences: a Different Kind of Data

A **sequence** is a list of observations whose **order carries information**: a clinical note, an ECG, a series of exposure measurements, a DNA strand.

*"no history of stroke"*      vs      *"history of stroke"*

Almost the same words. The opposite meaning.

# Sequences: a Different Kind of Data

A **sequence** is a list of observations whose **order carries information**: a clinical note, an ECG, a series of exposure measurements, a DNA strand.

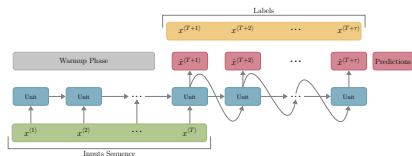
*“no history of stroke”*      vs      *“history of stroke”*

Almost the same words. The opposite meaning.

- ▶ A convolutional network exploits **spatial** structure; a recurrent network exploits **sequential** structure
- ▶ Notation:  $x = (x^{<1>}, x^{<2>}, \dots, x^{<T>})$ , each  $x^{<t>} \in \mathbb{R}^p$
- ▶ The length  $T$  **changes from one patient to the next**

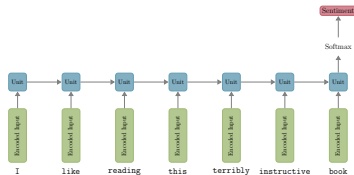
*A fully connected network needs a fixed-size input. A sequence has none.*

# What Do We Do With a Sequence?



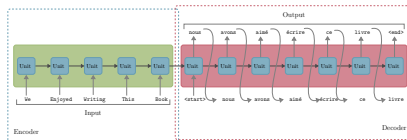
(a) predict what comes **next**

next word, next value of a time series

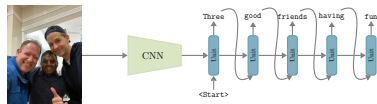


(b) **classify** a whole sequence

sentiment of a text, is this ECG abnormal?

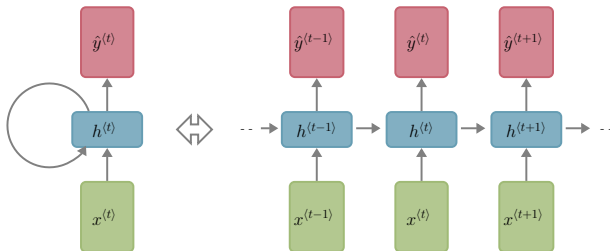


(c) **translate** one sequence into another  
machine translation, encoder and decoder

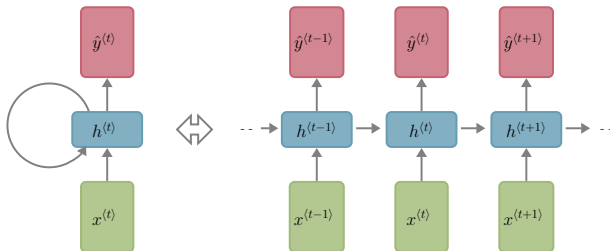


(d) turn something else **into** a sequence  
describing an image in words

# The Recurrent Idea: a Loop and a Memory



# The Recurrent Idea: a Loop and a Memory



- ▶ The network keeps an internal **state**  $h^{<t>}$ : its memory of everything read so far
- ▶ At each step, the state is updated from the previous state and the new input:

$$h^{<t>} = f(h^{<t-1>}, x^{<t>}), \quad h^{<0>} = 0$$

- ▶ The same function  $f$ , with the same weights, at every time step

*That is why a single model can read a sequence of any length  $T$ .*

# Inside the Unit

At each time step, the state and the output are computed as

$$\begin{cases} h^{<t>} = S_h(W_{hh} h^{<t-1>} + W_{hx} x^{<t>} + b_h) \\ \hat{y}^{<t>} = S_y(W_{yh} h^{<t>} + b_y) \end{cases}$$

# Inside the Unit

At each time step, the state and the output are computed as

$$\begin{cases} h^{<t>} = S_h(W_{hh} h^{<t-1>} + W_{hx} x^{<t>} + b_h) \\ \hat{y}^{<t>} = S_y(W_{yh} h^{<t>} + b_y) \end{cases}$$

With  $x^{<t>} \in \mathbb{R}^p$ ,  $h^{<t>} \in \mathbb{R}^m$ ,  $\hat{y}^{<t>} \in \mathbb{R}^q$ , the weights are

$$W_{hx} \in \mathbb{R}^{m \times p}, \quad W_{hh} \in \mathbb{R}^{m \times m}, \quad W_{yh} \in \mathbb{R}^{q \times m}$$

The same three matrices are reused at *every* step, whatever the length  $T$ .

# Inside the Unit

At each time step, the state and the output are computed as

$$\begin{cases} h^{<t>} = S_h(W_{hh} h^{<t-1>} + W_{hx} x^{<t>} + b_h) \\ \hat{y}^{<t>} = S_y(W_{yh} h^{<t>} + b_y) \end{cases}$$

With  $x^{<t>} \in \mathbb{R}^p$ ,  $h^{<t>} \in \mathbb{R}^m$ ,  $\hat{y}^{<t>} \in \mathbb{R}^q$ , the weights are

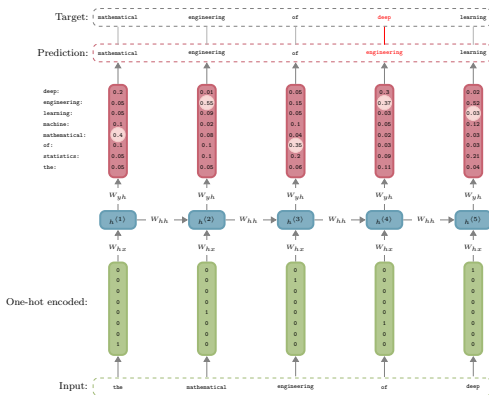
$$W_{hx} \in \mathbb{R}^{m \times p}, \quad W_{hh} \in \mathbb{R}^{m \times m}, \quad W_{yh} \in \mathbb{R}^{q \times m}$$

The same three matrices are reused at *every* step, whatever the length  $T$ .

A weighted sum, a bias, an activation: exactly the neuron we started from, with **one extra arrow coming from the past**.

# A Toy Example

A lexicon of  $p = 8$  words: each word is a **one-hot** vector of  $\mathbb{R}^8$ , read one at a time.



*At every step the model predicts the next word, and the prediction is compared with the target.*

In practice the one-hot is not fed directly: an **embedding layer** sits in between, learned together with the network, or initialised from word2vec / GloVe and fine-tuned.

# Training, and the Memory Problem

## Training

The principle is the one we saw earlier, **gradient descent**, applied to the unrolled network. In practice it is markedly harder and slower than for a feedforward network.

# Training, and the Memory Problem

## Training

The principle is the one we saw earlier, **gradient descent**, applied to the unrolled network. In practice it is markedly harder and slower than for a feedforward network.

## The problem

The same weight matrix is applied again and again, so the influence of the early inputs **fades away** as the sequence goes on. The model ends up learning from what is **nearby** only.

# Training, and the Memory Problem

## Training

The principle is the one we saw earlier, **gradient descent**, applied to the unrolled network. In practice it is markedly harder and slower than for a feedforward network.

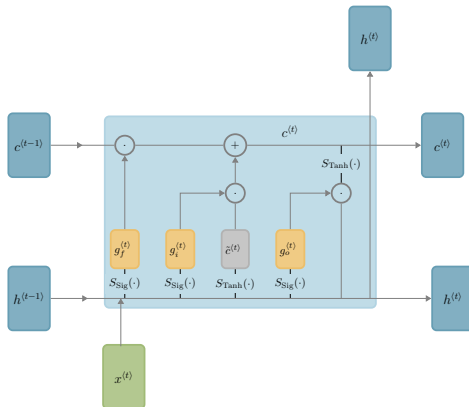
## The problem

The same weight matrix is applied again and again, so the influence of the early inputs **fades away** as the sequence goes on. The model ends up learning from what is **nearby** only.

*“Slava grew up in **Ukraine** before he moved around the world, first to the United States, and then to Australia. He loves teaching languages and is an avid teacher of his own mother tongue . . .”*

To fill in the blank, the model needs the **first word** of the text.

# LSTM: the Cell State Runs Straight Through



*Two states instead of one, and three gates controlling what passes.*

# LSTM: Two Memories and Three Gates

- ▶ Two states:  $c^{<t>}$  carries the **long term**,  $h^{<t>}$  the **short term**
- ▶ Three **gates**, vectors with entries between 0 and 1, each one open, closed, or in between: **forget**, **input**, **output**
- ▶ The cell state runs straight across the top of the picture, like a **conveyor belt**. The gates decide what is dropped, what is added, what is read

# LSTM: Two Memories and Three Gates

- ▶ Two states:  $c^{<t>}$  carries the **long term**,  $h^{<t>}$  the **short term**
- ▶ Three **gates**, vectors with entries between 0 and 1, each one open, closed, or in between: **forget**, **input**, **output**
- ▶ The cell state runs straight across the top of the picture, like a **conveyor belt**. The gates decide what is dropped, what is added, what is read

$$c^{<t>} = \underbrace{g_f \odot c^{<t-1>}}_{\text{what we keep}} + \underbrace{g_i \odot \tilde{c}^{<t>}}_{\text{what we add}}$$

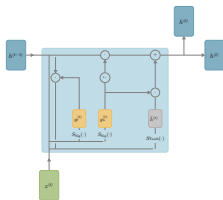
# LSTM: Two Memories and Three Gates

- ▶ Two states:  $c^{<t>}$  carries the **long term**,  $h^{<t>}$  the **short term**
- ▶ Three **gates**, vectors with entries between 0 and 1, each one open, closed, or in between: **forget**, **input**, **output**
- ▶ The cell state runs straight across the top of the picture, like a **conveyor belt**. The gates decide what is dropped, what is added, what is read

$$c^{<t>} = \underbrace{g_f \odot c^{<t-1>}}_{\text{what we keep}} + \underbrace{g_i \odot \tilde{c}^{<t>}}_{\text{what we add}}$$

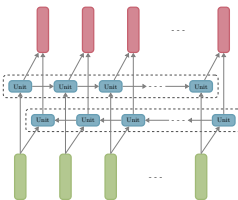
Information now travels along  $c$  by **additions**, instead of being multiplied again and again by the same matrix. That is why the memory survives.

# Three Common Variants



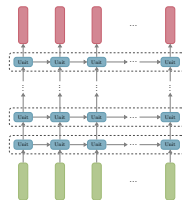
## GRU

a lighter LSTM:  
two gates instead of  
three



## Bidirectional

reads the sequence  
forwards *and* backwards



## Stacked

several recurrent layers  
on top of each other

# What RNNs and LSTMs Cannot Do

The hidden state  $h^{<t>}$  is a representation of a word in its context: the network does reshape the vectors. But it does so in a very constrained way.

# What RNNs and LSTMs Cannot Do

The hidden state  $h^{<t>}$  is a representation of a word in its context: the network does reshape the vectors. But it does so in a very constrained way.

- ▶ It reads **one step at a time**: nothing can be computed in parallel, so training on long sequences is slow
- ▶ The context flows in **one direction only**, left to right: a word is never informed by what comes *after* it
- ▶ A distant word reaches the present only by passing through **every intermediate step**, and the memory is a summary that is rewritten each time

# What RNNs and LSTMs Cannot Do

The hidden state  $h^{<t>}$  is a representation of a word in its context: the network does reshape the vectors. But it does so in a very constrained way.

- ▶ It reads **one step at a time**: nothing can be computed in parallel, so training on long sequences is slow
- ▶ The context flows in **one direction only**, left to right: a word is never informed by what comes *after* it
- ▶ A distant word reaches the present only by passing through **every intermediate step**, and the memory is a summary that is rewritten each time

What if every word could look at **every other word directly**, without travelling through the whole sentence?

# Do I still have your attention?

Now, let's see how machines focus on the **important information**, just like clinicians do.



"Attention is all you need" – Vaswani et al., 2017

# Attention: Selecting Relevant Information

## Human Attention

- ▶ When analysing imaging, clinicians focus on suspicious regions
- ▶ When reading clinical notes, doctors focus on key symptoms

## Key Idea

Attention = focusing on relevant information while ignoring distractions

# Attention: Selecting Relevant Information

## Human Attention

- ▶ When analysing imaging, clinicians focus on suspicious regions
- ▶ When reading clinical notes, doctors focus on key symptoms

## Key Idea

Attention = focusing on relevant information while ignoring distractions

## Example: Clinical Note

The patient reports a **headache** and took ibuprofen for arthritis.

## Clinical Question:

What is the patient's main symptom?

Model Needs To Focus On  
**Headache**

# Attention: Selecting Relevant Information

## Human Attention

- ▶ When analysing imaging, clinicians focus on suspicious regions
- ▶ When reading clinical notes, doctors focus on key symptoms

## Key Idea

Attention = focusing on relevant information while ignoring distractions

## Example: Clinical Note

The patient reports a **headache** and took ibuprofen for arthritis.

## Clinical Question:

What is the patient's main symptom?

Model Needs To Focus On  
**Headache**

**Attention helps models focus on the right words, just like clinicians do.**

# How Does a Model Use Attention?

**Clinical Question:** What is the patient's main symptom?

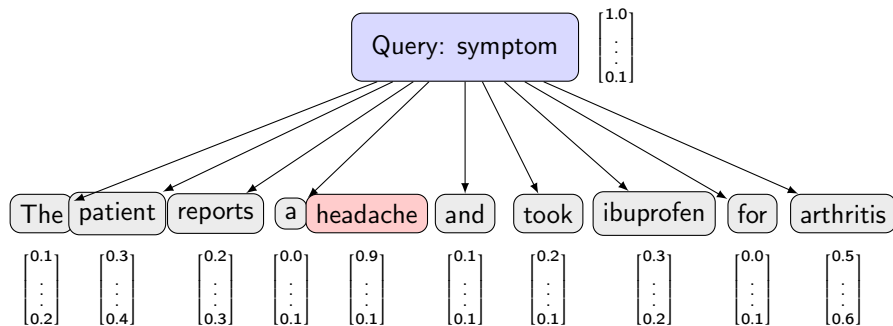
Query: symptom

$$\begin{bmatrix} 1.0 \\ \vdots \\ 0.1 \end{bmatrix}$$

|                                                      |                                                      |                                                      |                                                      |                                                      |                                                      |                                                      |                                                      |                                                      |                                                      |
|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|
| The                                                  | patient                                              | reports                                              | a                                                    | headache                                             | and                                                  | took                                                 | ibuprofen                                            | for                                                  | arthritis                                            |
| $\begin{bmatrix} 0.1 \\ \vdots \\ 0.2 \end{bmatrix}$ | $\begin{bmatrix} 0.3 \\ \vdots \\ 0.4 \end{bmatrix}$ | $\begin{bmatrix} 0.2 \\ \vdots \\ 0.3 \end{bmatrix}$ | $\begin{bmatrix} 0.0 \\ \vdots \\ 0.1 \end{bmatrix}$ | $\begin{bmatrix} 0.9 \\ \vdots \\ 0.1 \end{bmatrix}$ | $\begin{bmatrix} 0.1 \\ \vdots \\ 0.1 \end{bmatrix}$ | $\begin{bmatrix} 0.2 \\ \vdots \\ 0.1 \end{bmatrix}$ | $\begin{bmatrix} 0.3 \\ \vdots \\ 0.2 \end{bmatrix}$ | $\begin{bmatrix} 0.0 \\ \vdots \\ 0.1 \end{bmatrix}$ | $\begin{bmatrix} 0.5 \\ \vdots \\ 0.6 \end{bmatrix}$ |

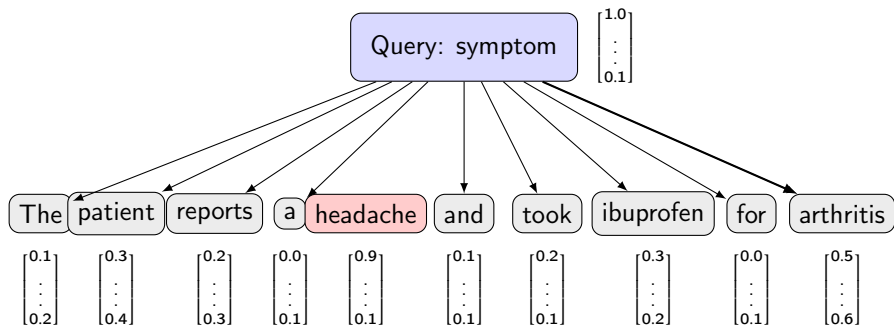
# How Does a Model Use Attention?

**Clinical Question:** What is the patient's main symptom?



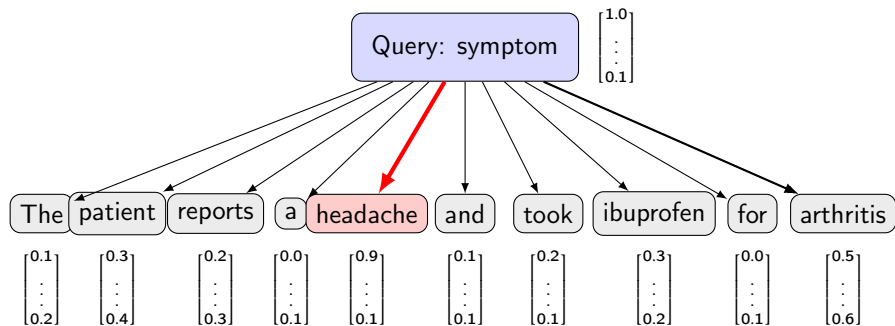
# How Does a Model Use Attention?

**Clinical Question:** What is the patient's main symptom?



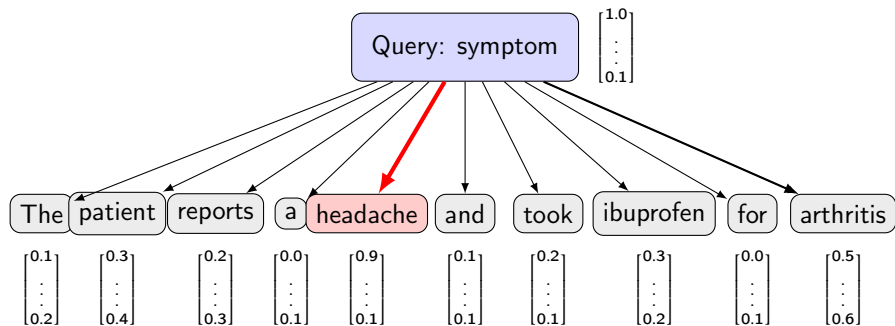
# How Does a Model Use Attention?

**Clinical Question:** What is the patient's main symptom?



# How Does a Model Use Attention?

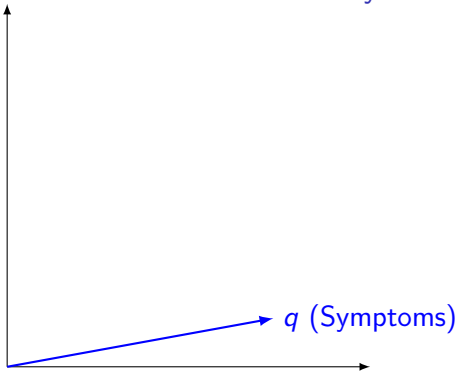
**Clinical Question:** What is the patient's main symptom?



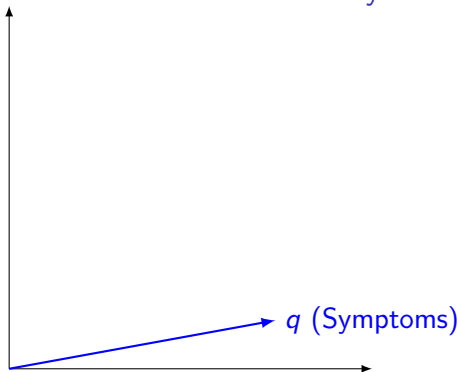
## Key Idea

The model compares the question with each word and assigns importance using **Dot Product**.

# Why Does Dot Product Measure Similarity?



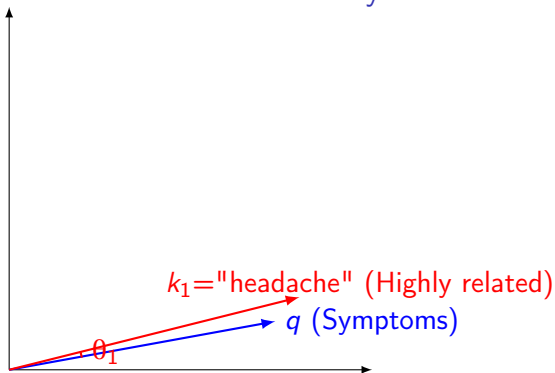
# Why Does Dot Product Measure Similarity?



$$q \cdot k = \overbrace{\|q\|}^{\text{Norm of } q} \times \underbrace{\|k\|}_{\text{Norm of } k} \times \cos(\theta)$$

- ▶ Small angle  $\rightarrow$  High similarity
- ▶  $90^\circ$  angle  $\rightarrow$  No similarity

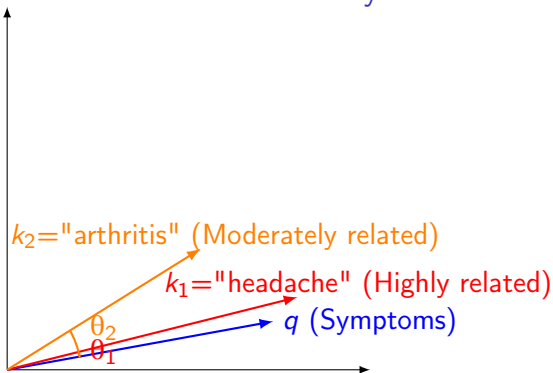
# Why Does Dot Product Measure Similarity?



$$q \cdot k = \overbrace{\|q\|}^{\text{Norm of } q} \times \underbrace{\|k\|}_{\text{Norm of } k} \times \cos(\theta)$$

- ▶ Small angle  $\rightarrow$  High similarity
- ▶  $90^\circ$  angle  $\rightarrow$  No similarity

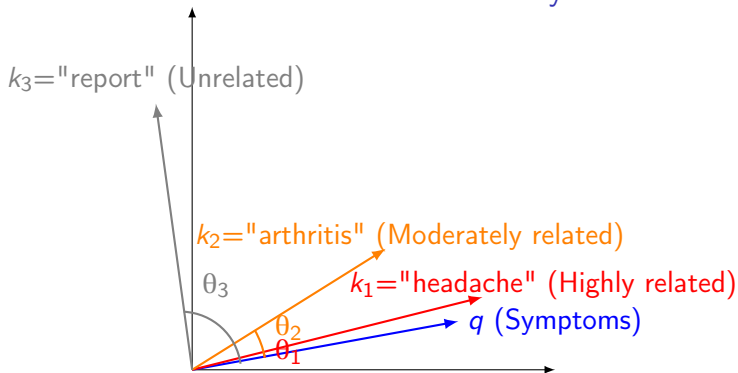
# Why Does Dot Product Measure Similarity?



$$q \cdot k = \overbrace{\|q\|}^{\text{Norm of } q} \times \underbrace{\|k\|}_{\text{Norm of } k} \times \cos(\theta)$$

- ▶ Small angle  $\rightarrow$  High similarity
- ▶  $90^\circ$  angle  $\rightarrow$  No similarity

# Why Does Dot Product Measure Similarity?



$$q \cdot k = \overbrace{\|q\|}^{\text{Norm of } q} \times \underbrace{\|k\|}_{\text{Norm of } k} \times \cos(\theta)$$

- ▶ Small angle  $\rightarrow$  High similarity
- ▶  $90^\circ$  angle  $\rightarrow$  No similarity

# From Similarity to Answer: Attention Mechanism

## Step 1: Compute Attention Weights

- ▶ Softmax converts these similarity scores into **weights**  $\alpha_i$  between 0 and 1
- ▶ Words more relevant to the question get higher weights

| Word ( $k_i$ )  | Weight $\alpha_i$ |
|-----------------|-------------------|
| The             | 0.02              |
| patient         | 0.05              |
| reports         | 0.03              |
| a               | 0.01              |
| <b>headache</b> | <b>0.75</b>       |
| and             | 0.02              |
| took            | 0.03              |
| ibuprofen       | 0.05              |
| for             | 0.02              |
| arthritis       | 0.02              |

# From Similarity to Answer: Attention Mechanism

## Step 1: Compute Attention Weights

- ▶ Softmax converts these similarity scores into **weights**  $\alpha_i$  between 0 and 1
- ▶ Words more relevant to the question get higher weights

| Word ( $k_i$ )  | Weight $\alpha_i$ |
|-----------------|-------------------|
| The             | 0.02              |
| patient         | 0.05              |
| reports         | 0.03              |
| a               | 0.01              |
| <b>headache</b> | <b>0.75</b>       |
| and             | 0.02              |
| took            | 0.03              |
| ibuprofen       | 0.05              |
| for             | 0.02              |
| arthritis       | 0.02              |

## Step 2: Build Context Vector

- ▶ Multiply each word vector by its attention weight  $\alpha_i$
- ▶ Sum weighted vectors to get the **context vector**:

$$c = \sum_i \alpha_i v_i$$

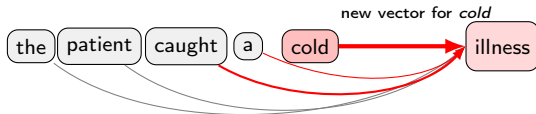
**Context Vector**  $c \rightarrow$  mostly influenced by **headache**

## Step 3: Model Output

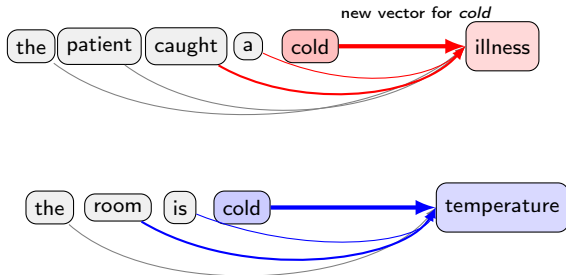
- ▶ Context vector  $c$  is fed into the model's prediction layer
- ▶ Produces the answer to the clinical question

**Main Symptom = Headache**

# Self-Attention: Every Word Queries the Others

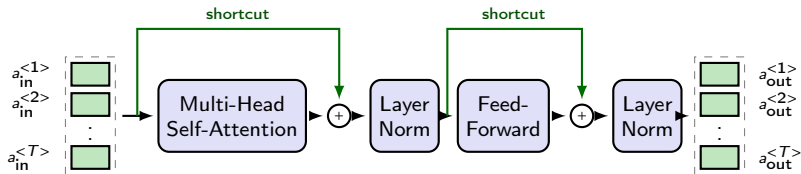


# Self-Attention: Every Word Queries the Others



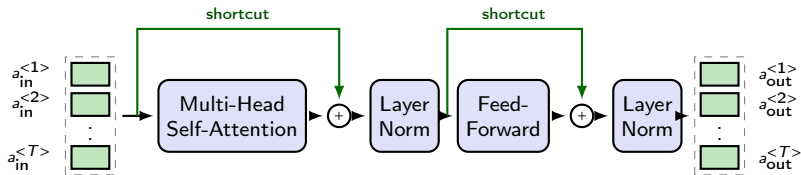


# Inside a Transformer Block



*$T$  vectors in,  $T$  vectors out, same dimension. The output sequence is the input of the next block: that is what lets them be stacked.*

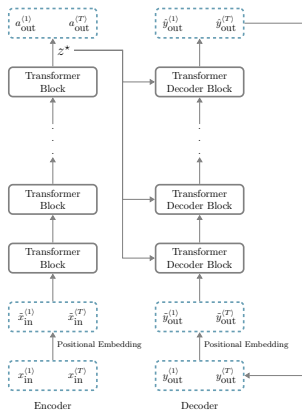
# Inside a Transformer Block



*$T$  vectors in,  $T$  vectors out, same dimension. The output sequence is the input of the next block: that is what lets them be stacked.*

- ▶ **Self-attention is the only step that mixes the positions.** The rest of the block treats each vector separately
- ▶ **Multi-head** means the attention is run **several times in parallel**, each copy free to look for a different kind of relation
- ▶ The **shortcuts** carry each vector forward unchanged, so every step only has to *adjust* it

# The Transformer



- ▶ One **block** is just two steps: **multi-head self-attention**, then a small **feedforward network**, each followed by a shortcut and a normalisation
- ▶ Stack the same block  $N$  times, add the positional embeddings at the entrance, and that is the whole architecture
- ▶ The **encoder** reads, the **decoder** writes. A generative model keeps the decoder alone
- ▶ A large language model is exactly this, made very large: GPT-3 has 96 blocks and 175 billion parameters

*The details are tomorrow morning.*

## Question 7?

*Do attention mechanisms allow a model to give different importance to various parts of the input??*

Thank you for your attention!

**Key takeaway:** The math behind these AI models is nothing new, just familiar concepts applied in clever ways.

Thank you for your attention!

**Key takeaway:** The math behind these AI models is nothing new, just familiar concepts applied in clever ways.

Question 8: *After this lecture, do neural networks feel less like a black box?*

# Any questions?

Thanks to my co-authors: **Sarat Moka** and **Yoni Nazarathy**

1. Liquet, B., Moka, S., & Nazarathy, Y. (2024). *Mathematical Engineering of Deep Learning, 1st Edition*. Available online: <https://deeplearningmath.org>
2. Liquet, B., Moka, S., Nazarathy, Y. (2024). Book chapter. *Navigating Mathematical Basics: A Primer for Deep Learning in Science* Available online: <https://deeplearningmath.org>